

UNIVERSIDADE DE SÃO PAULO

Instituto de Ciências Matemáticas e de Computação

Aplicando Neuroevolução em *Physics Informed Neural Networks* (PINNs)

Gabriel Milanez Cavalheri

Monografia - MBA em Inteligência Artificial e Big Data

SERVIÇO DE PÓS-GRADUAÇÃO DO ICMC-USP

Data de Depósito:

Assinatura: _____

Gabriel Milanez Cavalheri

Aplicando Neuroevolução em *Physics Informed Neural Networks* (PINNs)

Monografia apresentada ao Departamento de Ciências de Computação do Instituto de Ciências Matemáticas e de Computação, Universidade de São Paulo - ICMC/USP, como parte dos requisitos para obtenção do título de Especialista em Inteligência Artificial e Big Data.

Área de concentração: Inteligência Artificial

Orientador: Prof. Dr. Rodrigo Colnago Contreras

Versão revisada

São Carlos

2026

AUTORIZO A REPRODUÇÃO E DIVULGAÇÃO TOTAL OU PARCIAL DESTE TRABALHO, POR QUALQUER MEIO CONVENCIONAL OU ELETRÔNICO PARA FINS DE ESTUDO E PESQUISA, DESDE QUE CITADA A FONTE.

Ficha catalográfica elaborada pela Biblioteca Prof. Achille Bassi, ICMC/USP, com os dados fornecidos pelo(a) autor(a)

Cavalheri, Gabriel Milanez

Aplicando Neuroevolução em *Physics Informed Neural Networks* (PINNs) / Gabriel Milanez Cavalheri ; orientador Rodrigo Colnago Contreras. – São Carlos, 2026.

72 p. : il. (algumas color.) ; 30 cm.

Monografia (MBA em Inteligência Artificial e Big Data) – Instituto de Ciências Matemáticas e de Computação, Universidade de São Paulo, 2026.

1. Neuroevolução. 2. Algoritmos Genéticos. 3. PINN. 4. Redes Neurais. 5. Busca de Arquitetura Neural. I. Contreras, Rodrigo Colnago, orient. II. Título.

Gabriel Milanez Cavalheri

Applying Neuroevolution in *Physics Informed Neural Networks* (PINNs)

Monograph presented to the Departamento de Ciências de Computação do Instituto de Ciências Matemáticas e de Computação, Universidade de São Paulo - ICMC/USP, as part of the requirements for obtaining the title of Specialist in Artificial Intelligence and Big Data.

Concentration area: Artificial Intelligence

Advisor: Prof. Dr. Rodrigo Colnago Contreras

Revised version

São Carlos

2026

Este trabalho é dedicado à minha companheira e futura esposa Elaine, e à minha família, em especial aos meus pais: Leandro e Patricia, por sempre me apoiarem em todos os momentos e dificuldades, sem eles esse trabalho não existiria. Dedicado em especial ao meu orientador Rodrigo, que me direcionou e orientou nessa área tão interessante e atual, que são as PINNs.

AGRADECIMENTOS

Ao meu orientador, Prof. Dr. Rodrigo Colnago Contreras, pela orientação, pela confiança e pelas discussões que me apresentaram às *Physics Informed Neural Networks* e à neuroevolução, e que tornaram este trabalho possível.

À minha companheira e futura esposa, Elaine, pelo apoio, pela paciência e pelo incentivo em todas as etapas do curso.

Aos meus pais, Leandro e Patricia, e a toda a minha família, por sempre me apoiarem em todos os momentos e dificuldades.

Aos professores e colegas do MBA em Inteligência Artificial e Big Data do Instituto de Ciências Matemáticas e de Computação da Universidade de São Paulo (ICMC-USP), pelos conhecimentos compartilhados ao longo do curso.

“Um autor, em seu livro, deve ser como Deus no universo, presente em todos os lugares e invisível em lugar nenhum.”

Gustave Flaubert

RESUMO

Cavalheri, G. M. **Aplicando Neuroevolução em *Physics Informed Neural Networks* (PINNs)**. 2026. 72 p. Monografia (MBA em Inteligência Artificial e Big Data) - Instituto de Ciências Matemáticas e de Computação, Universidade de São Paulo, São Carlos, 2026.

A arquitetura de uma rede neural, isto é, o número de camadas ocultas e de neurônios por camada, afeta diretamente sua precisão e costuma ser escolhida por tentativa e erro, de forma custosa. Nas *Physics Informed Neural Networks* (PINNs), cada tentativa exige ainda resolver a equação diferencial que rege o problema. Este trabalho investiga se um procedimento leve de neuroevolução é capaz de selecionar arquiteturas de PINN melhores. Um Algoritmo Genético de uma única geração, baseado apenas em mutação, evolui o número de camadas ocultas e de neurônios por camada, enquanto os pesos são treinados com o otimizador Adam; as candidatas são ranqueadas por treinamentos parciais curtos, de modo que a busca custa cerca de 1,5 treinamento completo. Inicialmente, mostra-se no problema do pêndulo que, com apenas dez observações, uma rede neural treinada somente com o erro quadrático médio dos dados não extrapola a dinâmica do sistema, enquanto a PINN a recupera. Em seguida, o método é avaliado em três problemas clássicos com solução exata, o pêndulo simples, a transferência de calor em uma barra e a corda oscilante, em 35 rodadas reprodutíveis por problema, contra uma arquitetura sorteada e contra uma busca aleatória com o mesmo orçamento de épocas, usando o erro L^2 relativo em relação à solução exata. A arquitetura do Algoritmo Genético teve erro menor do que a arquitetura aleatória em 23, 21 e 26 das 35 rodadas, com razões medianas de erro de 4,6, 1,5 e 1,8; a melhora foi estatisticamente significativa no pêndulo e na corda oscilante (teste de Wilcoxon com correção de Holm, $p = 0,027$ e $0,003$), mas não na transferência de calor, que quase qualquer arquitetura resolveu. O Algoritmo Genético, porém, não superou a busca aleatória com o mesmo orçamento em nenhum dos problemas, o que indica que o benefício vem de avaliar várias candidatas com um indicador confiável, o custo do treinamento parcial, cuja correlação de Spearman com o erro variou de 0,79 a 0,99, e não dos operadores evolutivos.

Palavras-chave: Neuroevolução. Algoritmos Genéticos. PINN. Redes Neurais. Busca de Arquitetura Neural. Pêndulo. Equação da Onda. Equação do Calor.

ABSTRACT

Cavalheri, G. M. **Applying Neuroevolution in *Physics Informed Neural Networks* (PINNs)**. 2026. 72 p. Monograph (MBA in Artificial Intelligence and Big Data) - Instituto de Ciências Matemáticas e de Computação, Universidade de São Paulo, São Carlos, 2026.

The architecture of a neural network, i.e., its number of hidden layers and of neurons per layer, directly affects its accuracy and is usually chosen through costly trial and error. In Physics Informed Neural Networks (PINNs), every trial also requires solving the governing differential equation. This work investigates whether a lightweight neuroevolution procedure can select better PINN architectures. A single-generation, mutation-only Genetic Algorithm evolves the number of hidden layers and of neurons per layer, while the weights are trained with the Adam optimizer; the candidates are ranked through short partial trainings, so the search costs about 1.5 full trainings. First, the pendulum problem shows that, with only ten observations, a neural network trained only on the mean squared error of the data fails to extrapolate the dynamics, whereas the PINN recovers it. Then, the method is evaluated on three classical problems with exact solutions, the simple pendulum, heat conduction in a bar, and the vibrating string, in 35 reproducible runs per problem, against a randomly drawn architecture and against random search with the same epoch budget, using the relative L^2 error against the exact solution. The architecture of the Genetic Algorithm had a lower error than the random architecture in 23, 21, and 26 of the 35 runs, with median error ratios of 4.6, 1.5, and 1.8; the improvement was statistically significant for the pendulum and the vibrating string (Wilcoxon test with Holm correction, $p = 0.027$ and 0.003), but not for heat conduction, which almost any architecture solved. However, the Genetic Algorithm did not outperform random search with the same budget in any problem, which indicates that the benefit comes from evaluating several candidates with a reliable proxy, the partial-training loss, whose Spearman correlation with the error ranged from 0.79 to 0.99, rather than from the evolutionary operators.

Keywords: Neuroevolution. Genetic Algorithms. PINN. Neural Networks. Neural Architecture Search. Pendulum. Wave Equation. Heat Equation.

LISTA DE FIGURAS

Figura 1 – Fluxograma do processo de neuroevolução da arquitetura da PINN. . .	42
Figura 2 – Solução exata do pêndulo, dados de treinamento e pontos de colocação da função de custo física.	56
Figura 3 – Valores preditos pela rede neural padrão após 200 e 1.000 épocas de treinamento.	57
Figura 4 – Valores preditos pela PINN após 6.000, 12.000, 18.000 e 20.000 épocas de treinamento.	58
Figura 5 – Funções de custo da rede neural padrão e das parcelas da função de custo da PINN, em função das épocas de treinamento.	59
Figura 6 – Solução exata do problema de transferência de calor utilizado neste trabalho: perfis de temperatura em cinco instantes e mapa de $u(x, t)$. .	61
Figura 7 – Solução exata do problema da corda oscilante utilizado neste trabalho: perfis de deslocamento em cinco instantes e mapa de $u(x, t)$	62
Figura 8 – Erro L^2 relativo da rede final de cada braço nas 35 rodadas de cada problema, em escala logarítmica. As caixas mostram os quartis, e cada ponto é uma rodada.	63
Figura 9 – PINN obtida com a arquitetura do GA na rodada de erro mediano de cada problema. Acima: pêndulo, com as dez observações. Meio e abaixo: solução exata, predição da PINN e erro absoluto na transferência de calor e na corda oscilante.	64
Figura 10 – Função de custo total ao longo do treinamento completo dos três braços na rodada de erro mediano do GA em cada problema. As linhas finas mostram o custo em cada época, e as grossas, sua mediana móvel em 200 épocas. As legendas indicam as arquiteturas como camadas ocultas $\times$ neurônios por camada.	67

LISTA DE TABELAS

Tabela 1	– Configuração dos experimentos, idêntica para os três problemas.	54
Tabela 2	– As 5 arquiteturas selecionadas na triagem da rodada 5 do pêndulo, com sua <i>fitness</i> após E_{fit} épocas, o descendente gerado por mutação e a <i>fitness</i> de ambos após E_{term} épocas. As arquiteturas são indicadas como (camadas ocultas, neurônios por camada).	59
Tabela 3	– Arquiteturas dos três braços na rodada 5 do pêndulo e erro L^2 relativo da PINN após o treinamento completo.	60
Tabela 4	– Mediana do erro L^2 relativo e mediana do número de parâmetros treináveis da rede final de cada braço (35 rodadas por problema), e correlação de Spearman ρ_S entre a <i>fitness</i> e o erro L^2 relativo das 350 candidatas triadas pelo GA em cada problema.	63
Tabela 5	– Comparações pareadas do GA com cada referência nas 35 rodadas de cada problema, no erro L^2 relativo. Vitórias: rodadas em que o GA obteve erro menor, com o intervalo de Wilson de 95% da taxa de vitória; p_B : teste binomial unilateral; p_W : teste de postos sinalizados de Wilcoxon unilateral; ambos ajustados por Holm sobre as seis comparações. Razão mediana: mediana do erro da referência dividido pelo erro do GA, com intervalo <i>bootstrap</i> de 95% (valores acima de 1 favorecem o GA). Vitórias (custo): vitórias na métrica secundária.	65

LISTA DE ABREVIATURAS E SIGLAS

CE	Computação Evolucionária
CNN	Convolutional Neural Network
EDO	Equação Diferencial Ordinária
EDP	Equação Diferencial Parcial
GA	Genetic Algorithms
GPU	Graphics Processing Unit
GRU	Gated Recurrent Unit
LSTM	Long Short-Term Memory
MAE	Mean Absolute Error
MLP	Multilayer Perceptron
MSE	Mean Squared Error
NEAT	NeuroEvolution of Augmenting Topologies
PINN	Physics Informed Neural Network
PSO	Particle Swarm Optimization
ReLU	Rectified Linear Unit
SGD	Stochastic Gradient Descent

SUMÁRIO

1	INTRODUÇÃO	25
2	FUNDAMENTOS MATEMÁTICOS	29
2.1	Redes Neurais	29
2.2	PINN	31
2.3	Neuroevolução	34
2.3.1	Computação Evolucionária e Algoritmos Genéticos	34
2.3.2	Neuroevolução	35
2.3.3	Representação do Indivíduo	36
2.3.4	Função de Fitness	37
2.3.5	Operadores Evolutivos	38
2.3.6	Algoritmo de Neuroevolução	40
2.3.7	Métodos de Neuroevolução na Literatura	41
2.3.8	Neuroevolução e Treinamento Baseado em Gradiente	43
2.3.9	Complexidade e Limitações	44
2.3.10	Neuroevolução Aplicada a Este Trabalho	45
3	TRABALHOS CORRELATOS	47
3.1	Physics Informed Neural Networks	47
3.2	Neuroevolução em Redes Neurais e PINNs	47
4	METODOLOGIA	49
4.1	Visão Geral do Método	49
4.2	Algoritmo Genético	49
4.3	Estratégia de Validação	50
4.3.1	Braços de Comparação	50
4.3.2	Métricas	50
4.3.3	Análise Estatística	50
4.4	Reprodutibilidade	51
5	AVALIAÇÃO EXPERIMENTAL	53
5.1	Setup Computacional	53
5.2	Configuração da Neuroevolução	53
5.3	Pêndulo	55
5.3.1	Rede Neural Padrão e PINN	55
5.3.2	Exemplo de uma Rodada de Neuroevolução	58
5.4	Transferência de calor em uma barra unidimensional	60

5.5	Corda oscilante	61
5.6	Resultados da Neuroevolução	62
5.6.1	GA versus Arquitetura Aleatória	65
5.6.2	GA versus Busca Aleatória com o Mesmo Orçamento	65
5.6.3	O Treinamento Parcial como Indicador	66
5.6.4	Arquiteturas Seleccionadas e Custo Computacional	66
5.7	Discussão	68
6	CONCLUSÕES	69
	REFERÊNCIAS	71

1 INTRODUÇÃO

As redes neurais artificiais (Géron, 2019) são ferramentas estatísticas muito poderosas para resolver problemas de regressão e classificação quando se possui dados sobre o problema. É muito comum em problemas físicos relacionados com a equação do calor, como a transferência de calor em uma barra, com a equação da onda, como a corda oscilante, e com oscilações, como o pêndulo (Thornton; Marion, 2004), a realização de muitas medições durante a fase de experimentação, para depois facilitar a análise e extrapolação dos dados obtidos. Porém, também é muito comum que durante a coleta de dados, várias variáveis interfiram em uma coleta de dados sistemática e padronizada. Isso resulta em dados faltantes e com diferentes intervalos de coleta.

As redes neurais geralmente utilizam apenas uma função de custo, que tem o objetivo de minimizar o erro entre valor esperado e o valor predito pelo modelo, nas etapas de treinamento e validação do modelo. A função de custo costuma funcionar bem para problemas gerais, com muitos dados espaçados.

O presente trabalho aborda três cenários extremamente ineficientes para redes neurais convencionais, problemas físicos que evoluem no tempo e com poucos dados iniciais (Brigato; Iocchi, 2021). Com esse cenário, não importa o tempo de treinamento do modelo ou o tamanho da sua arquitetura, os valores preditos não irão generalizar e extrapolar os dados ausentes.

A arquitetura de rede neural baseada em informações físicas, *Physics Informed Neural Networks* (PINN) (Raissi; Perdikaris; Karniadakis, 2019), consegue generalizar, extrapolar e se adaptar aos poucos dados já coletados, isso ocorre devido às informações físicas previamente adicionadas na função de custo da rede neural.

A escolha da arquitetura de uma rede neural artificial, o número de camadas e de neurônios em cada camada, é um dos principais desafios no desenvolvimento desses modelos, pois impacta diretamente seu desempenho e sua capacidade de generalização. Essa escolha é comumente definida com base em análises experimentais exaustivas, que podem ser extremamente custosas em tempo e recursos computacionais. Por exemplo, o tamanho do principal modelo Llama 3 (Grattafiori *et al.*, 2024) foi definido a partir de experimentos de leis de escala, nos quais foram pré-treinados modelos de 40 milhões a 16 bilhões de parâmetros, e apenas o pré-treinamento do modelo final, de 405 bilhões de parâmetros, utilizou até 16 mil GPUs, com um recorte de 54 dias desse treinamento relatado pelos autores. Esse cenário levanta uma questão central: como garantir que a melhor arquitetura foi escolhida sem que seja necessário realizar o treinamento completo do modelo para cada configuração testada?

A Neuroevolução (*Neuroevolution*) (Akut; Kulkarni, 2019) é o conceito de otimização de estruturas em redes neurais, que pode identificar a melhor arquitetura, sem a necessidade de todo o treinamento da rede neural artificial. Utilizam-se metaheurísticas ou estratégias evolutivas, por exemplo, Algoritmos Genéticos, ou do inglês, *Genetic Algorithms* (Holland, 1992) para selecionar os melhores atributos durante a fase inicial e intermediária do treinamento. Este conceito baseia-se na seleção de características desejadas de indivíduos (Stanley *et al.*, 2019), no caso do presente trabalho foram as arquiteturas das PINNs, com o intuito de obter um resultado melhor do que apenas selecionar uma arquitetura aleatória.

O objetivo deste trabalho é aplicar a neuroevolução à escolha da arquitetura de PINNs e avaliar, de forma reprodutível, se ela seleciona arquiteturas melhores do que uma escolha aleatória e do que uma busca aleatória com o mesmo orçamento computacional. Para isso, propõe-se um Algoritmo Genético de uma única geração, baseado apenas em mutação, que evolui o número de camadas ocultas e de neurônios por camada da PINN, enquanto os pesos continuam sendo treinados por gradiente descendente, e que avalia cada candidata por meio de um treinamento parcial, de modo que a busca completa custa cerca de 1,5 treinamento completo. O método é avaliado em três problemas físicos clássicos com solução exata, o pêndulo simples, a transferência de calor em uma barra e a corda oscilante, em 35 rodadas reprodutíveis por problema, usando como métrica principal o erro da PINN em relação à solução exata. Busca-se responder a três perguntas: (i) a arquitetura escolhida pelo Algoritmo Genético é mais precisa do que uma arquitetura escolhida aleatoriamente? (ii) os operadores evolutivos acrescentam valor em relação a simplesmente avaliar o mesmo número de candidatas sorteadas, com o mesmo orçamento de épocas? e (iii) o custo de um treinamento parcial é um bom indicador da qualidade final de uma arquitetura?

Sobre esta versão revisada. Após o depósito da versão original desta monografia, identificou-se que as redes dos experimentos de transferência de calor e de corda oscilante utilizavam a função de ativação ReLU. Como uma rede com ReLU é linear por partes em relação às suas entradas, suas derivadas segundas são nulas, e os termos $\partial^2 u / \partial x^2$ e $\partial^2 u / \partial t^2$ das equações desses problemas não eram representados pela PINN. Nesta versão, todos os experimentos foram redesenhados e refeitos: todas as redes utilizam a tangente hiperbólica, os problemas foram formulados de forma adimensional, com solução exata conhecida, o erro em relação a essa solução passou a ser a métrica principal, todas as decisões aleatórias passaram a usar sementes fixas, e acrescentou-se a busca aleatória com o mesmo orçamento como segunda referência. Os resultados apresentados nos Capítulos 5 e 6 substituem os da versão original, e o código e os resultados completos estão disponíveis publicamente¹.

Este trabalho está organizado da seguinte forma: o Capítulo 2 apresenta os funda-

¹ <https://github.com/GMCavaleri/MBA-PINN-Neuroevolution>

mentos matemáticos necessários, abordando redes neurais e PINN; o Capítulo 3 discute os trabalhos correlatos, revisando a literatura existente sobre PINNs e sobre a aplicação de neuroevolução na otimização de redes neurais e PINNs; o Capítulo 4 descreve a metodologia utilizada, incluindo as ferramentas empregadas e a configuração dos experimentos realizados para os três problemas físicos abordados; o Capítulo 5 apresenta e discute os resultados obtidos na avaliação experimental; por fim, o Capítulo 6 traz as conclusões do trabalho.

2 FUNDAMENTOS MATEMÁTICOS

Este capítulo apresenta os conceitos teóricos necessários para a compreensão deste trabalho. Inicialmente, na Seção 2.1, revisam-se os fundamentos de redes neurais artificiais, com base principalmente na obra de Géron (2019). Em seguida, a Seção 2.2 apresenta o conceito de PINN, introduzido por Raissi, Perdikaris e Karniadakis (2019) e posteriormente consolidado pelo *survey* de Karniadakis *et al.* (2021) e pela revisão de Cuomo *et al.* (2022), que mapeiam o estado da arte e os principais desafios em aberto dessa abordagem. Por fim, a Seção 2.3 apresenta os fundamentos de computação evolucionária e Algoritmos Genéticos (Holland, 1992; Goldberg, 1989), o conceito de neuroevolução e suas diferentes formas de aplicação a redes neurais (Yao, 1999), e a formalização do método de neuroevolução empregado neste trabalho para otimizar a arquitetura das PINNs. Destaca-se que a combinação entre neuroevolução e PINNs ainda é pouco explorada na literatura, sendo o *survey* de Wong *et al.* (2026) uma das poucas referências que discute especificamente estratégias evolutivas aplicadas à otimização de PINNs, reforçando a relevância do tema abordado nesta monografia.

2.1 Redes Neurais

Uma rede neural totalmente conectada, também conhecida como *Multilayer Perceptron* (MLP) (Géron, 2019), é composta de vários neurônios, denominados *Perceptrons*, divididos em camadas interconectadas. Divide-se uma MLP em 3 tipos de camadas: entrada, ocultas e saída. De forma geral, a saída de cada camada l pode ser descrita por

$$a^{(l)} = f \left(W^{(l)} a^{(l-1)} + b^{(l)} \right), \quad (2.1)$$

em que $a^{(l-1)}$ é o vetor de saída da camada anterior (sendo $a^{(0)} = x$ o vetor de entrada da rede), $W^{(l)}$ e $b^{(l)}$ são, respectivamente, a matriz de pesos e o vetor de *bias* da camada l , e f é a função de ativação.

A camada de entrada deve possuir um neurônio para cada atributo de entrada dos dados. As camadas ocultas variam muito dependendo da arquitetura da rede, conectando a camada de entrada à camada de saída; seu número pode variar de uma a várias camadas, e sua principal função, além de conectar os dados que entram e saem da rede, é aprender e armazenar conhecimento por meio de sucessivas interações com os dados de treinamento. Já a camada de saída deve possuir um neurônio para cada atributo alvo, ou seja, para cada valor que se deseja prever (Géron, 2019).

O processo de aprendizado é possível porque cada neurônio possui parâmetros livres, também conhecidos como pesos sinápticos (*weights*), que não são fixos e possuem a

capacidade de variar em contato com novos dados, também denominados sinais de entrada x_n (*inputs*). Os sinais de entrada são multiplicados por seus respectivos pesos sinápticos w_n e somados em uma função linear z (*linear function*), junto com uma constante b (*bias*) (Géron, 2019):

$$z = w^T x + b. \quad (2.2)$$

O valor resultante da função linear é utilizado como parâmetro por outra função, denominada função de ativação (*step function*), que transforma o valor da soma em um valor de saída $h_{w,b}(x)$ (*output*):

$$h_{w,b}(x) = \text{step}(z) = \text{step}(w^T x + b). \quad (2.3)$$

Uma das métricas mais utilizadas para avaliar a qualidade do aprendizado de uma rede neural é a função de custo (*loss function*), também chamada de função objetivo, que quantifica o erro entre os valores previstos pela rede e os valores esperados para cada amostra do conjunto de dados. O objetivo do processo de treinamento é encontrar o conjunto de parâmetros (pesos e *bias*) que minimize essa função.

Existem diversas funções de custo na literatura, cada uma mais adequada a determinado tipo de problema. Para problemas de regressão, como os abordados neste trabalho, uma das mais utilizadas é o Erro Quadrático Médio (*Mean Squared Error*, MSE), definido como

$$MSE = \frac{1}{N} \sum_{i=1}^N (\bar{x}_i - x_i)^2, \quad (2.4)$$

em que N é o número total de amostras consideradas, e $\bar{x}_i$ e x_i são, respectivamente, o valor predito e o valor esperado para a i -ésima amostra. Além do MSE, outras funções de custo também são amplamente empregadas, como o Erro Absoluto Médio (*Mean Absolute Error*, MAE), mais robusto à presença de *outliers*, e a Entropia Cruzada (*Cross-Entropy*), comumente utilizada em problemas de classificação.

A minimização da função de custo é realizada por meio de algoritmos de otimização baseados no gradiente descendente (*gradient descent*), que ajustam iterativamente os parâmetros da rede na direção oposta ao gradiente da função de custo em relação a esses parâmetros. Esse princípio de aprendizado por correção do erro remonta ao Adaline (*Adaptive Linear Neuron*), proposto por Widrow e Hoff (1960), um dos primeiros modelos de neurônio artificial treinado por meio da regra Delta, que ajusta os pesos proporcionalmente ao erro entre a saída prevista e a saída desejada:

$$e_k(n) = \bar{x}_k(n) - x_k(n), \quad (2.5)$$

em que n é o índice da amostra dentro do conjunto de dados de treinamento, e k é o índice do neurônio da camada de saída correspondente. Esse mesmo princípio de ajuste baseado no erro é a base do algoritmo de retropropagação (*backpropagation*), utilizado atualmente para treinar redes neurais de múltiplas camadas por meio do gradiente descendente.

A rede aprende quando o seguinte procedimento ocorre: os sinais de entrada se propagam por toda a rede, alterando os parâmetros livres dos neurônios e gerando sinais de saída no domínio dos atributos alvos. A cada iteração desse processo, o valor da função de custo tende a diminuir e se aproximar de zero. Conforme a função de custo se aproxima de zero, os valores previstos tendem a interpolar os dados de treinamento; porém, se o modelo não conseguir manter um valor baixo para a função de custo quando exposto a dados de teste, ocorre o fenômeno de *overfitting*, no qual o modelo não consegue generalizar para novos dados.

A escolha da função de ativação f (Equação 2.1) também influencia o que a rede é capaz de representar. Duas das mais utilizadas são a tangente hiperbólica, $f(z) = \tanh(z)$, e a unidade linear retificada (*Rectified Linear Unit*, ReLU), $f(z) = \max(0, z)$ (Géron, 2019). A ReLU é barata de calcular e atenua o desaparecimento do gradiente em redes profundas, o que a tornou padrão em muitas aplicações. Contudo, por ser linear por partes, uma rede composta apenas por camadas com ReLU é também uma função linear por partes de suas entradas: sua derivada primeira é constante por partes, e sua derivada segunda é nula em quase todo ponto. A tangente hiperbólica, por outro lado, é suave e possui derivadas de todas as ordens. Essa diferença é irrelevante quando a rede é treinada apenas com dados, mas é decisiva nas PINNs, apresentadas a seguir, cuja função de custo envolve derivadas da rede em relação às suas entradas.

2.2 PINN

Diversos fenômenos físicos são descritos por equações diferenciais, que relacionam uma função desconhecida às suas derivadas. Quando essa função depende de uma única variável independente, a equação é denominada Equação Diferencial Ordinária (EDO); quando depende de duas ou mais variáveis independentes, como o tempo e uma ou mais coordenadas espaciais, e envolve derivadas parciais em relação a essas variáveis, a equação é denominada Equação Diferencial Parcial (EDP).

De forma geral, seguindo a formulação apresentada por Raissi, Perdikaris e Karniadakis (2019), o problema físico a ser resolvido pode ser formulado da seguinte maneira: seja $u(x)$ a solução desconhecida de interesse, definida em um domínio $\Omega \subset \mathbb{R}^D$, com $x = (x_1, \dots, x_D)$ representando as variáveis independentes do problema, por exemplo:

posição e tempo. Deseja-se encontrar u tal que

$$\mathcal{N}[u](x) = 0, \quad x \in \Omega, \quad (2.6)$$

sujeita a condições de contorno

$$\mathcal{B}[u](x) = 0, \quad x \in \partial\Omega, \quad (2.7)$$

em que $\mathcal{N}$ é um operador diferencial, linear ou não linear, que pode envolver derivadas de u de qualquer ordem, Ω é o domínio do problema e $\partial\Omega$ representa sua fronteira, onde as condições de contorno são impostas.

Tradicionalmente, uma rede neural treinada para aproximar $u(x)$ utiliza apenas uma função de custo baseada em dados observados, como o MSE (Equação 2.4), que penaliza a diferença entre os valores previstos pela rede e os valores medidos ou conhecidos de u em um conjunto finito de pontos. Contudo, essa abordagem não garante que a solução aprendida satisfaça a Equação 2.6 fora dos pontos de treinamento, o que compromete sua capacidade de generalização e extrapolação, especialmente quando há poucos dados disponíveis.

A abordagem de PINN, proposta por Raissi, Perdikaris e Karniadakis (2019), soluciona esse problema ao incorporar o próprio operador diferencial $\mathcal{N}$ na função de custo da rede. Nessa abordagem, a solução $u(x)$ é aproximada por uma rede neural $\hat{u}(x; \theta)$, parametrizada pelos pesos θ da rede. Além dos dados observados, seleciona-se um conjunto de pontos de colocação (*collocation points*) $\{x_c\}_{c=1}^{N_c} \subset \Omega$, amostrados no domínio do problema, nos quais o resíduo da equação diferencial é calculado:

$$r(x) = \mathcal{N}[\hat{u}](x; \theta). \quad (2.8)$$

O resíduo $r(x)$ mede o quanto a solução aproximada $\hat{u}$ deixa de satisfazer a Equação 2.6 no ponto x : quanto mais próximo de zero, mais fielmente $\hat{u}$ obedece à equação diferencial naquele ponto, sendo um resíduo exatamente nulo o caso em que a rede satisfaz a física do problema exatamente em x . Como $\mathcal{N}$ pode envolver derivadas de $\hat{u}$ de qualquer ordem em relação às suas entradas, o cálculo de $r(x)$ depende dessas derivadas, obtidas por meio de diferenciação automática (*automatic differentiation*), disponível nativamente em bibliotecas de aprendizado profundo, como o PyTorch.

O resíduo calculado nos pontos de colocação é então agregado em uma segunda função de custo, denominada função de custo física, definida como o erro quadrático médio do resíduo sobre esses pontos:

$$\mathcal{L}_{\text{física}}(\theta) = \frac{1}{N_c} \sum_{c=1}^{N_c} r(x_c)^2. \quad (2.9)$$

Diferentemente da função de custo dos dados, que penaliza o desvio da rede em relação a valores observados de u , a função de custo física não depende de nenhum dado medido: ela penaliza exclusivamente o quanto a solução aprendida deixa de satisfazer a equação diferencial nos pontos de colocação amostrados, funcionando como uma forma de regularização baseada na física do problema. O número de pontos de colocação N_c utilizado, e sua distribuição no domínio Ω , variam conforme o problema físico considerado, sendo detalhados no Capítulo 5 para cada um dos três casos estudados neste trabalho.

A função de custo total da PINN é dada pela soma da função de custo dos dados ($\mathcal{L}_{\text{dados}}$, geralmente o MSE) e da função de custo física ($\mathcal{L}_{\text{física}}$, Equação 2.9):

$$\mathcal{L}_{\text{total}}(\theta) = \mathcal{L}_{\text{dados}}(\theta) + \mathcal{L}_{\text{física}}(\theta). \quad (2.10)$$

As condições iniciais e de contorno do problema (Equação 2.7) são impostas da mesma forma, como parcelas adicionais de erro quadrático médio, avaliadas em pontos amostrados no instante inicial e nas fronteiras do domínio; nesse caso, dizemos que são impostas de forma fraca (*soft constraints*), pois são apenas penalizadas, e não garantidas pela construção da rede. Quando as parcelas possuem escalas muito diferentes, a função de custo física pode ainda ser multiplicada por um peso $\lambda > 0$, de modo a equilibrar sua contribuição com a das demais parcelas.

Como o resíduo $r(x)$ envolve derivadas de $\hat{u}$ em relação às entradas, a rede precisa ser suficientemente suave para representá-las. Em particular, se a equação contém derivadas segundas, como $\partial^2 u / \partial x^2$ ou $\partial^2 u / \partial t^2$, uma rede com ativação ReLU não as representa, pois suas derivadas segundas são nulas em quase todo ponto (Seção 2.1): o termo correspondente simplesmente desaparece do resíduo, e a PINN passa a resolver uma equação diferente da desejada. Por esse motivo, todas as redes deste trabalho utilizam a tangente hiperbólica como função de ativação.

Ao minimizar essa função de custo combinada, a rede é forçada a encontrar uma solução que não apenas se ajuste aos dados observados, mas também respeite a física descrita pela equação diferencial em todo o domínio, permitindo que o modelo generalize e extrapole corretamente para regiões onde não há dados disponíveis. Essa característica torna as PINNs particularmente adequadas para problemas com poucos dados iniciais, como os três problemas físicos abordados neste trabalho, cujas formulações específicas e a forma particular assumida pelo operador $\mathcal{N}$ em cada caso, são apresentadas no Capítulo 5.

2.3 Neuroevolução

2.3.1 Computação Evolucionária e Algoritmos Genéticos

A Computação Evolucionária (CE) é uma família de técnicas de busca e otimização estocásticas, baseadas em populações de soluções candidatas, inspiradas nos mecanismos da evolução natural: seleção natural, reprodução e variação genética (Eiben; Smith, 2015). Sob esse guarda-chuva encontram-se diferentes paradigmas, desenvolvidos de forma relativamente independente ao longo da segunda metade do século XX, como os Algoritmos Genéticos, as Estratégias Evolutivas, a Programação Evolutiva e a Programação Genética. Apesar de suas diferenças de representação e de operadores, todos compartilham a mesma estrutura conceitual. Um conjunto de soluções candidatas é avaliado quanto à sua qualidade; novas soluções são então geradas iterativamente a partir das melhores soluções da geração anterior, por meio de operadores de variação e seleção (Eiben; Smith, 2015).

Este trabalho concentra-se especificamente nos Algoritmos Genéticos (*Genetic Algorithms*, GA), formalizados por Holland (1992) e sistematizados por Goldberg (1989) como um método de busca que representa cada solução candidata como uma cadeia de símbolos, análoga aos cromossomos biológicos, sobre a qual atuam operadores inspirados na genética Mendeliana. Antes de descrever o procedimento do GA, é importante estabelecer com precisão a terminologia empregada, uma vez que o uso impreciso desses termos é uma fonte comum de confusão na literatura aplicada:

- Indivíduo: uma solução candidata completa para o problema que se deseja resolver.
- Cromossomo (ou genótipo): a representação codificada de um indivíduo, isto é, a estrutura de dados sobre a qual os operadores evolutivos atuam diretamente.
- Gene: cada componente ou posição elementar do cromossomo, correspondente a um aspecto codificável da solução.
- Alelo: o valor específico assumido por um gene em um indivíduo particular.
- População: o conjunto de indivíduos mantidos e evoluídos simultaneamente em uma mesma geração.
- *Fitness* (ou aptidão): medida numérica da qualidade de um indivíduo em relação ao problema, utilizada para guiar o processo de seleção.

É importante destacar que um indivíduo corresponde à solução completa. Ele não deve ser confundido com um gene, que representa apenas um componente elementar dessa solução. Essa distinção, retomada na Seção 2.3.3 para o caso específico de redes neurais, é essencial para compreender como diferentes aspectos de uma solução podem ser codificados e evoluídos de forma independente.

De forma geral, o procedimento de um GA pode ser dividido em seis etapas (Holland, 1992; Goldberg, 1989): inicialização da população, avaliação da *fitness*, seleção, cruzamento e mutação, substituição e verificação do critério de parada. Na inicialização, gera-se aleatoriamente uma população inicial de indivíduos. Na avaliação da *fitness*, cada indivíduo é submetido a testes específicos do problema, obtendo-se uma medida de sua qualidade. Na seleção, os indivíduos com melhor *fitness* são escolhidos para gerar descendentes, enquanto os demais tendem a ser descartados. No cruzamento e mutação, novos indivíduos são gerados a partir dos selecionados, combinando (cruzamento) e/ou alterando aleatoriamente (mutação) seu material genético, na expectativa de que os descendentes possam superar o desempenho de seus antecessores. Na substituição, os descendentes tomam o lugar dos indivíduos de pior desempenho, mantendo constante o tamanho da população. Por fim, o critério de parada determina quando o processo evolutivo é encerrado, geralmente após um número fixo de gerações ou quando a *fitness* da população estabiliza.

Essa estrutura genérica, entretanto, não é suficiente para descrever como um GA pode ser utilizado para otimizar redes neurais: questão central deste trabalho, discutida a partir da Seção 2.3.2.

2.3.2 Neuroevolução

Neuroevolução é o nome dado à aplicação de técnicas de computação evolucionária, sobretudo Algoritmos Genéticos, ao projeto e/ou ao treinamento de redes neurais artificiais (Stanley *et al.*, 2019; Yao, 1999). O treinamento de uma rede neural pode ser entendido como um problema de otimização: busca-se, no espaço de configurações possíveis da rede, aquela que minimize (ou maximize) uma medida de desempenho sobre um conjunto de dados. Essa formulação é exatamente o que um algoritmo evolutivo resolve de forma genérica, tratando cada configuração candidata como um indivíduo e avaliando sua qualidade por meio de uma função de *fitness*. Diferentemente do gradiente descendente, um GA não exige que a função a ser otimizada seja diferenciável em relação aos parâmetros evoluídos, o que o torna adequado para espaços de busca discretos ou combinatórios, como o número de camadas ou de neurônios de uma arquitetura (Yao, 1999).

Segundo o levantamento de Yao (1999), a neuroevolução pode atuar sobre diferentes aspectos de uma rede neural, isoladamente ou em combinação:

- Pesos sinápticos: o indivíduo codifica diretamente os valores dos pesos e vieses da rede, sendo a arquitetura mantida fixa. A *fitness* é obtida executando a rede resultante sobre os dados do problema.
- Arquitetura (topologia): o indivíduo codifica características estruturais da rede, como o número de camadas ocultas e o número de neurônios por camada. Os pesos, nesse

caso, são tipicamente obtidos por um algoritmo de treinamento convencional, como o gradiente descendente, aplicado a cada arquitetura candidata.

- Hiperparâmetros: o indivíduo codifica parâmetros do próprio processo de treinamento, como taxa de aprendizado, tamanho de lote ou coeficientes de regularização.
- Funções de ativação: o indivíduo codifica a escolha da função de ativação de cada camada ou neurônio.
- Combinações desses elementos: diferentes aspectos podem ser evoluídos simultaneamente pelo mesmo indivíduo.

Uma distinção particularmente relevante é a que existe entre evoluir apenas os pesos de uma arquitetura fixa e evoluir simultaneamente pesos e topologia. No primeiro caso, o espaço de busca tem dimensionalidade fixa e igual ao número de parâmetros da rede; essa abordagem dispensa a necessidade de gradientes, mas tende a escalar mal com o aumento do número de parâmetros, tornando-se pouco competitiva frente ao gradiente descendente em redes profundas (Yao, 1999). No segundo caso, tanto a estrutura quanto os valores dos parâmetros evoluem ao longo das gerações; o método *NeuroEvolution of Augmenting Topologies* (NEAT), proposto por Stanley e Miikkulainen (2002), é o exemplo mais influente dessa categoria, sendo retomado na Seção 2.3.7.

Este trabalho situa-se em uma terceira categoria, intermediária às duas anteriores: evolui-se exclusivamente a arquitetura da rede, número de camadas ocultas e número de neurônios por camada, mantendo os pesos fora do processo evolutivo. Os pesos de cada arquitetura candidata continuam sendo obtidos por meio de treinamento baseado em gradiente, com o otimizador Adam (Kingma; Ba, 2014), conforme detalhado no Capítulo 4. Essa abordagem, também empregada por Akut e Kulkarni (2019) para o projeto automático de arquiteturas de aprendizado profundo, evita o custo de evoluir diretamente um espaço de pesos de altíssima dimensionalidade, concentrando o esforço evolutivo exclusivamente na escolha estrutural da rede.

2.3.3 Representação do Indivíduo

Conforme discutido na Seção 2.3.2, a forma como um indivíduo é codificado depende de quais aspectos da rede neural estão sendo evoluídos. Duas formas de codificação ilustram bem esse ponto.

Quando o objetivo é evoluir os pesos de uma arquitetura fixa, o cromossomo costuma ser um vetor real, obtido pela concatenação de todas as matrizes de pesos $W^{(l)}$ e vetores de vies $b^{(l)}$ da rede (Equação 2.1), achatados em uma única sequência de números. Nesse caso, cada gene corresponde a um único peso ou vies, e o alelo é o valor real assumido por esse parâmetro.

Quando o objetivo é evoluir a arquitetura da rede, como neste trabalho, o cromossomo codifica características estruturais em vez de valores numéricos de pesos. Um indivíduo A_i pode ser representado como

$$A_i = (L_i, n_i), \quad (2.11)$$

em que L_i é o número de camadas ocultas da arquitetura, e n_i é o número de neurônios de cada camada oculta. Todas as camadas ocultas de uma arquitetura possuem o mesmo número de neurônios; as camadas de entrada e de saída, por sua vez, são definidas pelo problema físico, com um neurônio para cada variável de entrada e de saída. Nesse caso, cada gene corresponde a uma característica estrutural da rede: o primeiro gene é o número de camadas ocultas, e o segundo é o número de neurônios por camada oculta; o alelo é o valor inteiro assumido por esse gene em um indivíduo particular. Por exemplo, uma arquitetura com três camadas ocultas de 32 neurônios cada é codificada pelo cromossomo $(3, 32)$; o alelo do gene correspondente ao número de neurônios é, nesse indivíduo, igual a 32.

Vale notar que, embora os valores de L_i e n_i possam variar entre indivíduos diferentes, o cromossomo definido pela Equação 2.11 tem comprimento fixo, igual a 2, para todos os indivíduos. Essa representação compacta reduz o espaço de busca a pares (L_i, n_i) dentro dos limites definidos na Seção 5.2, à custa de não contemplar arquiteturas com números de neurônios diferentes entre as camadas ocultas.

2.3.4 Função de Fitness

A função de *fitness* é o elemento que conecta o algoritmo evolutivo ao problema que se deseja resolver. Ela atribui a cada indivíduo um valor numérico que representa sua qualidade, e é justamente esse valor que orienta a seleção: indivíduos com *fitness* melhor têm maior chance de gerar descendentes, enquanto os demais tendem a ser descartados ao longo das gerações.

A definição concreta da *fitness* depende do tipo de problema e do aspecto da rede que está sendo evoluído (Seção 2.3.2). Em problemas de classificação, é comum utilizar a acurácia ou a entropia cruzada sobre um conjunto de validação. Em problemas de regressão, como os abordados neste trabalho, é comum utilizar o erro quadrático médio (Equação 2.4) ou outra medida de erro entre os valores previstos e os valores esperados. De forma mais geral, quando a rede neural é utilizada para resolver um problema de otimização, como uma equação diferencial, a *fitness* pode ser definida diretamente como o valor da função objetivo desse problema.

Essa escolha também determina se o algoritmo deve maximizar ou minimizar a *fitness*. Medidas de qualidade, como a acurácia, devem ser maximizadas: quanto maior o

valor, melhor o indivíduo. Medidas de erro ou de custo, como o MSE, devem ser minimizadas: quanto menor o valor, melhor o indivíduo. Ambas as formulações são equivalentes entre si, bastando inverter o critério utilizado na etapa de seleção; a escolha de qual delas adotar é, em geral, uma questão de conveniência matemática ou de consistência com a métrica já empregada no treinamento da rede.

Neste trabalho, adota-se a segunda formulação. Para cada arquitetura candidata A_i (Equação 2.11), treina-se a PINN correspondente, com pesos θ_i , por um número reduzido de épocas E_{fit} , correspondente a uma fração do total de épocas de treinamento de uma PINN. Como o valor da função de custo oscila de uma época para outra durante o treinamento com o Adam, por vezes com picos de várias ordens de grandeza, define-se a *fitness* de A_i como a média da função de custo total da PINN (Equação 2.10) nas últimas W épocas desse treinamento parcial:

$$f(A_i) = \frac{1}{W} \sum_{e=E_{fit}-W+1}^{E_{fit}} \mathcal{L}_{\text{total}}^{(e)}(\theta_i), \quad (2.12)$$

em que $\mathcal{L}_{\text{total}}^{(e)}$ é o valor da função de custo total na época e . A *fitness* usa apenas a função de custo da PINN e nunca a solução exata do problema, que, em uma aplicação real, seria desconhecida. Como valores menores de f indicam melhor desempenho, o algoritmo busca minimizar a *fitness* de cada indivíduo. O uso de um treinamento parcial, em vez do treinamento completo, é o que torna viável avaliar múltiplas arquiteturas candidatas: treinar cada uma delas com o total de E_{total} épocas de treinamento tornaria o custo computacional da busca evolutiva proibitivo, aproximando-o do custo de uma busca exaustiva sobre o espaço de arquiteturas.

2.3.5 Operadores Evolutivos

Os operadores evolutivos são os mecanismos responsáveis por conduzir a população em direção a soluções melhores ao longo das gerações. De forma geral, distinguem-se quatro operadores: seleção, cruzamento (ou recombinação), mutação e substituição; o elitismo, quando presente, não constitui um operador à parte, mas uma regra aplicada durante a seleção ou a substituição.

Seleção é o operador que determina quais indivíduos da população atual participarão da reprodução. Entre as estratégias mais comuns na literatura estão a seleção por roleta, na qual a probabilidade de um indivíduo ser escolhido é proporcional à sua *fitness*; a seleção por torneio, na qual pequenos subconjuntos de indivíduos competem entre si e o de melhor *fitness* vence; e a seleção por truncamento, na qual apenas os indivíduos com as melhores *fitness* são mantidos, descartando-se o restante da população (Eiben; Smith, 2015; Goldberg, 1989). Este trabalho adota a seleção por truncamento: a população inicial

P_0 é ordenada pela *fitness*, em ordem crescente, e seleciona-se o subconjunto com os K melhores indivíduos:

$$P_0^{select} = \{A_{(1)}, \dots, A_{(K)}\}, \quad f(A_{(1)}) \leq f(A_{(2)}) \leq \dots \leq f(A_{(K)}). \quad (2.13)$$

Cruzamento (ou recombinação) é o operador que combina o material genético de dois indivíduos selecionados para gerar um ou mais descendentes, na expectativa de que a combinação preserve características vantajosas de ambos os pais. Em um cromossomo de comprimento fixo, uma forma comum é o cruzamento de um ponto: dado um ponto de corte k , o descendente herda os genes anteriores a k de um pai e os genes posteriores a k do outro. Esse operador, no entanto, não é utilizado neste trabalho. Com o cromossomo de dois genes definido pela Equação 2.11, o cruzamento se limitaria a combinar o número de camadas de um pai com o número de neurônios por camada do outro. Este trabalho opta por uma reprodução baseada exclusivamente em mutação, de modo que cada descendente seja uma pequena variação de uma das melhores arquiteturas da triagem. Essa escolha mantém o procedimento simples e a busca concentrada em torno das candidatas selecionadas, à custa de uma exploração do espaço de busca mais conservadora, já que cada descendente deriva de um único indivíduo, não da combinação de dois.

Mutação é o operador que introduz variação na população por meio de alterações aleatórias no cromossomo de um indivíduo. Em um GA clássico com cromossomo binário, por exemplo, cada gene é invertido com uma pequena probabilidade fixa, chamada taxa de mutação. Neste trabalho, a mutação segue uma lógica distinta: em vez de alterar cada gene com uma dada probabilidade, aplica-se, com probabilidade 1, um incremento aleatório não negativo a cada gene do indivíduo selecionado, gerando um único descendente por indivíduo. Formalmente, a partir de P_0^{select} , geram-se K descendentes por meio de um operador de mutação μ aplicado a cada arquitetura selecionada:

$$A'_i = \mu(A_i) = (L_i + \Delta L_i, n_i + \Delta n_i), \quad (2.14)$$

em que ΔL_i e Δn_i são incrementos aleatórios não negativos, sorteados a partir de distribuições uniformes discretas com limites pré-definidos, aplicados respectivamente ao número de camadas ocultas e ao número de neurônios por camada. Assim, o descendente pode ganhar uma camada oculta ou nenhuma, e o número de neurônios de todas as suas camadas ocultas, incluindo a camada eventualmente adicionada, aumenta na mesma quantidade ou se mantém; os valores concretos desses limites são apresentados na Seção 5.2. Como os incrementos são sempre não negativos, a mutação tende a gerar arquiteturas iguais ou maiores que as originais, nunca menores.

Substituição é o operador que define como os descendentes gerados passam a integrar a população seguinte. Entre as estratégias comuns estão a substituição geracional, na qual

toda a população é trocada pelos descendentes, e a substituição em regime estacionário (*steady-state*), na qual poucos indivíduos são substituídos por vez. Este trabalho adota uma substituição elitista: os K indivíduos selecionados em P_0^{select} permanecem inalterados, enquanto os descendentes tomam o lugar dos K indivíduos de pior desempenho da população original, mantendo constante o tamanho M da população e resultando na população final P_1 :

$$P_1 = P_0^{select} \cup \{A'_1, \dots, A'_K\}. \quad (2.15)$$

Nota-se que essa estratégia de substituição já incorpora uma forma de elitismo: como P_0^{select} contém exatamente os K melhores indivíduos de P_0 , e esses indivíduos são preservados sem alteração em P_1 , garante-se que a *fitness* do melhor indivíduo não piora entre a população inicial e a população final. Vale reforçar que P_0 e P_1 denotam, respectivamente, a população inicial e a população resultante da única geração executada pelo Algoritmo 1 (Seção 2.3.6), e não um índice genérico de geração g : como discutido adiante, este método não itera por múltiplas gerações.

2.3.6 Algoritmo de Neuroevolução

Reunindo os elementos definidos nas seções anteriores (representação do indivíduo, função de *fitness* e operadores evolutivos), o Algoritmo 1 descreve o procedimento de neuroevolução empregado neste trabalho para a busca da arquitetura da PINN.

A Figura 1 ilustra esse mesmo procedimento na forma de um fluxograma, tornando mais explícita a transição entre a população inicial P_0 , avaliada com E_{fit} épocas apenas para triagem, e a população final P_1 , reavaliada com E_{term} épocas antes da escolha de A^* .

Uma diferença importante em relação ao GA clássico descrito na Seção 2.3.1 merece destaque: o Algoritmo 1 não itera o ciclo de seleção, mutação e substituição por múltiplas gerações até a convergência da população. Em vez disso, executa uma única geração, avaliando a população inicial P_0 com um número reduzido de épocas E_{fit} apenas para triagem, e reavaliando a população resultante P_1 com um número maior de épocas E_{term} para obter uma medida mais confiável de *fitness* antes da escolha final. Essa opção de projeto decorre do custo computacional de treinar uma PINN: diferentemente de problemas em que a *fitness* é barata de calcular e permite milhares de gerações, cada avaliação aqui exige treinar uma rede neural por um número não trivial de épocas, o que torna um GA de muitas gerações computacionalmente inviável para este trabalho. Essa questão é retomada com mais detalhe na Seção 2.3.9.

Ao final do Algoritmo 1, a arquitetura A^* é treinada por completo e comparada a duas arquiteturas de referência, conforme descrito nas Seções 2.3.10 e 4.3.

Algorithm 1 Neuroevolução da arquitetura da PINN

Require: Tamanho da população M ; número de indivíduos selecionados K ; épocas de avaliação E_{fit} ; épocas de reavaliação E_{term} ; limites de camadas ocultas e de neurônios por camada

Ensure: Arquitetura selecionada A^*

- 1: Gerar população inicial $P_0 = \{A_1, \dots, A_M\}$ aleatoriamente, respeitando os limites de camadas e neurônios
 - 2: **for** cada indivíduo $A_i \in P_0$ **do**
 - 3: Treinar a PINN de arquitetura A_i por E_{fit} épocas
 - 4: Calcular $f(A_i)$ (Equação 2.12)
 - 5: **end for**
 - 6: Ordenar P_0 por *fitness* crescente e obter P_0^{select} , os K melhores indivíduos (Equação 2.13)
 - 7: **for** cada indivíduo $A_i \in P_0^{select}$ **do**
 - 8: Gerar descendente $A'_i = \mu(A_i)$ por mutação (Equação 2.14)
 - 9: **end for**
 - 10: Formar a nova população $P_1 = P_0^{select} \cup \{A'_1, \dots, A'_K\}$ (Equação 2.15)
 - 11: **for** cada indivíduo $A_i \in P_1$ **do**
 - 12: Treinar a PINN de arquitetura A_i por E_{term} épocas
 - 13: Calcular $f(A_i)$ (Equação 2.12)
 - 14: **end for**
 - 15: $A^* \leftarrow \arg \min_{A_i \in P_1} f(A_i)$
 - 16: **return** A^*
-

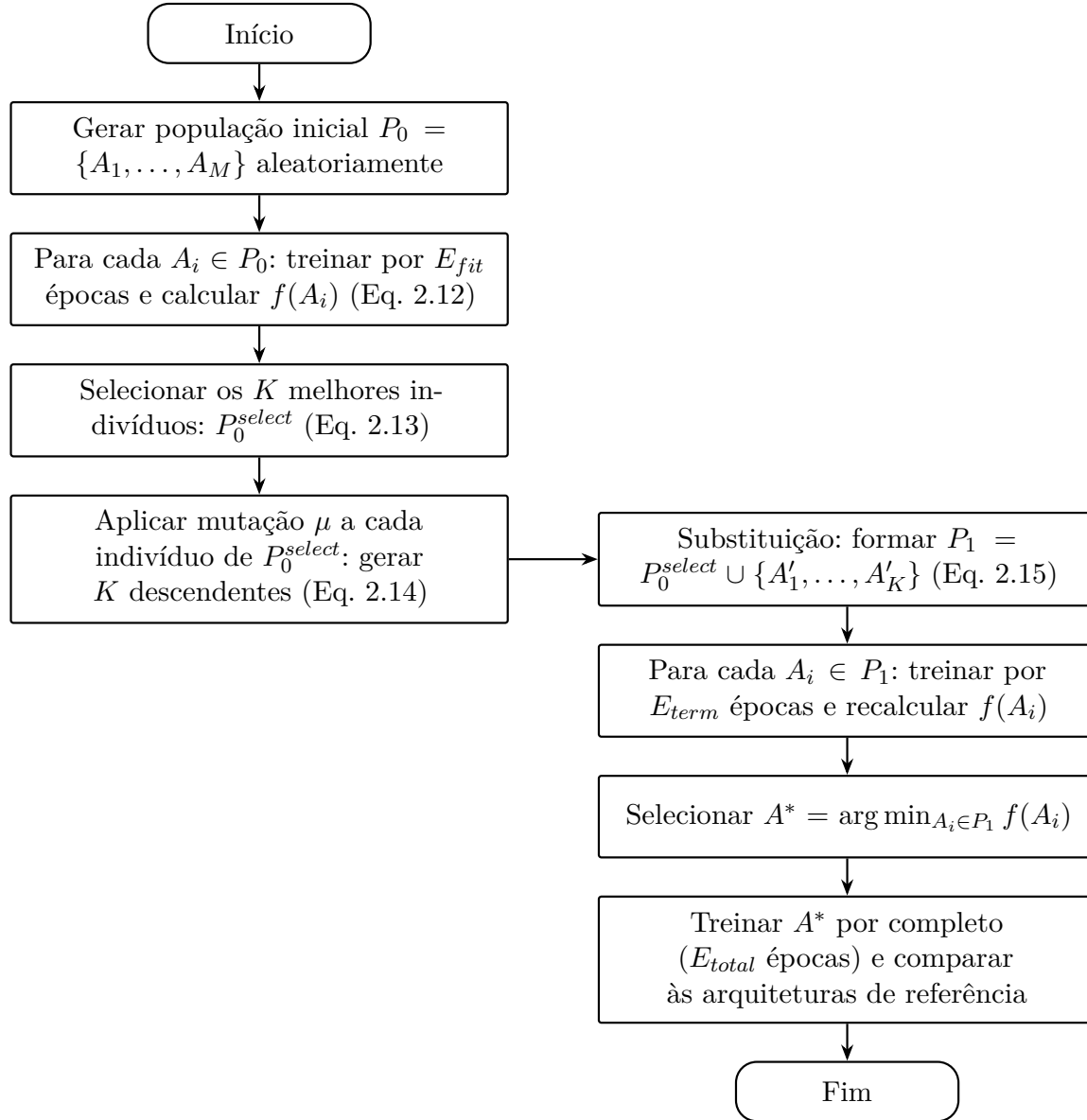
2.3.7 Métodos de Neuroevolução na Literatura

Como discutido na Seção 2.3.2, os métodos de neuroevolução propostos na literatura podem ser agrupados, de forma geral, conforme o aspecto da rede neural que evoluem: pesos, topologia, ou ambos simultaneamente.

Os primeiros trabalhos em neuroevolução concentraram-se na evolução direta dos pesos de arquiteturas fixas, tratando o treinamento da rede como um problema de otimização a ser resolvido por um algoritmo evolutivo, em vez de pelo gradiente descendente. O levantamento de Yao (1999) cataloga extensivamente essa linha de pesquisa, observando que, embora competitiva em redes pequenas ou em problemas nos quais o gradiente não está disponível, essa abordagem escala mal com o aumento do número de parâmetros, perdendo competitividade frente ao gradiente descendente em redes de maior porte.

Entre os métodos que evoluem a topologia da rede, o mais influente é o NEAT, proposto por Stanley e Miikkulainen (2002). A motivação do NEAT parte de uma crítica ao projeto manual de arquiteturas: em vez de fixar a topologia da rede a priori e evoluir apenas seus pesos, ou de buscar em um espaço de arquiteturas definido manualmente, o NEAT evolui pesos e topologia simultaneamente, partindo de redes minimais, sem camadas ocultas, e complexificando-as gradualmente por meio de mutações estruturais que adicionam nós e conexões. Essa estratégia de complexificação progressiva evita explorar, desde o início,

Figura 1 – Fluxograma do processo de neuroevolução da arquitetura da PINN.



Fonte: Elaborada pelo autor.

um espaço de arquiteturas desnecessariamente grande. Para viabilizar esse processo, o NEAT introduz dois mecanismos centrais. O primeiro é a marcação histórica dos genes por meio de números de inovação, que permite identificar genes de origem comum entre indivíduos com topologias distintas, tornando possível um cruzamento coerente entre eles, o que resolve o problema das convenções competitivas (*competing conventions*), isto é, a dificuldade de recombinar de forma consistente redes com topologias diferentes. O segundo é a especiação, que agrupa indivíduos topologicamente semelhantes em subpopulações e compartilha a *fitness* entre os membros de cada espécie, protegendo inovações estruturais recentes da competição imediata com o restante da população, até que seus pesos tenham oportunidade de ser otimizados.

Nem todos os métodos de evolução de topologia recorrem à complexificação incremental e à especiação do NEAT. Uma alternativa mais simples, adotada por Akut e Kulkarni (2019) para o projeto automático de arquiteturas de aprendizado profundo, consiste em codificar diretamente características estruturais de alto nível, como o número de camadas e de neurônios por camada, em um cromossomo de tamanho reduzido, e buscar por meio de um GA convencional a combinação que produz o melhor desempenho, sem incorporar complexificação progressiva, especiação ou histórico de inovações.

O método empregado neste trabalho, detalhado na Seção 2.3.10, aproxima-se dessa segunda categoria: evolui-se diretamente o número de camadas ocultas e de neurônios por camada, sem os mecanismos de especiação e de marcação histórica do NEAT, e sem cruzamento, apenas mutação, conforme discutido na Seção 2.3.5. Trabalhos que aplicam neuroevolução especificamente à otimização de PINNs, incluindo o próprio problema de busca de arquitetura tratado nesta monografia, são discutidos separadamente no Capítulo 3.

2.3.8 Neuroevolução e Treinamento Baseado em Gradiente

O treinamento de uma rede neural por gradiente descendente, descrito na Seção 2.1, depende de a função de custo ser diferenciável em relação aos parâmetros que se deseja otimizar, uma vez que o algoritmo de retropropagação calcula justamente o gradiente dessa função em relação a esses parâmetros. Essa exigência é satisfeita pelos pesos e vieses de uma arquitetura fixa, mas não por escolhas estruturais como o número de camadas ocultas ou o número de neurônios por camada: não existe um gradiente da função de custo em relação a uma quantidade discreta como "número de camadas", já que alterar esse valor muda a própria estrutura do grafo computacional da rede, não apenas o valor de um parâmetro contínuo dentro dele. É justamente essa limitação que torna o gradiente descendente inaplicável à busca de arquitetura, e que motiva o uso de um método como a neuroevolução, que não exige diferenciabilidade em relação aos parâmetros evoluídos (Yao, 1999).

Além de lidar naturalmente com parâmetros discretos, a busca evolutiva mantém e avalia, a cada geração, uma população inteira de soluções candidatas, em vez de seguir uma única trajetória guiada pelo gradiente a partir de um ponto de partida. Essa característica permite explorar simultaneamente diferentes regiões do espaço de busca, o que pode ser vantajoso em espaços com múltiplos ótimos locais ou em problemas nos quais a relação entre a escolha estrutural e a qualidade final da rede não é suave. No contexto específico das PINNs, Sung *et al.* (2023) mostram, comparando algoritmos de neuroevolução ao gradiente descendente estocástico, que os métodos evolutivos podem inclusive superar o gradiente descendente quanto ao grau de conformidade da solução aprendida com a equação diferencial que rege o problema físico, conforme discutido com mais detalhe no

Capítulo 3.

Essas vantagens, no entanto, têm um custo. Cada avaliação de *fitness* em um algoritmo de neuroevolução exige treinar, ainda que parcialmente, a rede correspondente a cada indivíduo da população, o que é ordens de magnitude mais caro do que uma única atualização de pesos por retropropagação. Enquanto o gradiente descendente aproveita a informação de direção fornecida pelo gradiente para convergir de forma direta a um mínimo local da função de custo, uma busca evolutiva aplicada ao espaço de pesos precisa explorá-lo de forma essencialmente cega, sem informação de direção, o que a torna pouco competitiva para otimizar diretamente os pesos de redes com muitos parâmetros, como discutido na Seção 2.3.7.

É exatamente essa complementaridade, e não uma relação de substituição, que orienta a escolha metodológica deste trabalho: o gradiente descendente, por meio do otimizador Adam (Kingma; Ba, 2014), permanece responsável por treinar os pesos de cada arquitetura candidata, aproveitando sua eficiência em espaços de parâmetros contínuos e diferenciáveis; a neuroevolução é reservada exclusivamente para a busca da arquitetura, um espaço discreto e não diferenciável no qual o gradiente descendente não pode ser aplicado diretamente.

2.3.9 Complexidade e Limitações

Ao contrário do treinamento por gradiente descendente, cujo custo por iteração é aproximadamente constante e conhecido de antemão, o custo computacional de um algoritmo de neuroevolução é dominado pelo número de avaliações de *fitness* realizadas, cada uma delas exigindo o treinamento, ainda que parcial, de uma rede neural distinta. No Algoritmo 1, a população inicial P_0 , com M indivíduos, é avaliada uma vez com E_{fit} épocas; a população resultante da substituição, P_1 , também com M indivíduos, é avaliada novamente com E_{term} épocas. O custo total da busca evolutiva corresponde, portanto, a $2M$ treinamentos parciais de PINN, um número consideravelmente menor do que o exigido por uma busca exaustiva, que treinaria por completo cada uma das arquiteturas candidatas consideradas ao longo do processo.

Esse custo é diretamente proporcional ao tamanho da população M e ao número de indivíduos selecionados K : aumentar M amplia a diversidade de arquiteturas exploradas na inicialização, ao custo de mais avaliações de *fitness*; aumentar K preserva mais indivíduos da população original e gera mais descendentes por geração, ao custo de reduzir a pressão seletiva, já que uma fração maior da população sobrevive independentemente de seu desempenho relativo. Os valores concretos adotados para M e K neste trabalho são apresentados na Seção 5.2.

A noção clássica de convergência de um GA, associada à estabilização da *fitness* da população ao longo de muitas gerações (Eiben; Smith, 2015), não se aplica diretamente

ao método empregado neste trabalho, uma vez que, como discutido na Seção 2.3.6, apenas uma única geração de seleção e mutação é executada, por razões de custo computacional. Isso significa que o método não busca convergir a população a uma região específica do espaço de arquiteturas ao longo do tempo; realiza, em vez disso, um único refinamento sobre a população inicial, cuja qualidade depende fortemente de essa população inicial já conter, por sorteio aleatório, arquiteturas razoavelmente promissoras.

Essa limitação se relaciona diretamente à diversidade da população. Como o operador de mutação definido pela Equação 2.14 aplica apenas incrementos não negativos, a busca tende a explorar unicamente arquiteturas iguais ou maiores que as da população inicial, nunca menores, introduzindo um viés direcional na exploração do espaço de busca. Além disso, como os descendentes são gerados exclusivamente a partir dos K melhores indivíduos de P_0 , e o método não incorpora mecanismos de preservação de diversidade, como a especiação empregada pelo NEAT (Seção 2.3.7), a exploração fica restrita a uma vizinhança relativamente estreita desses indivíduos. Diferentemente do risco clássico de convergência prematura, em que a diversidade se perde ao longo de muitas gerações, o risco aqui é mais imediato: por haver uma única geração e nenhuma reintrodução de diversidade, uma população inicial pouco representativa do espaço de arquiteturas pode comprometer a qualidade da arquitetura final A^* , sem que o algoritmo tenha oportunidade de se recuperar dessa escolha ao longo de gerações subsequentes.

2.3.10 Neuroevolução Aplicada a Este Trabalho

Esta seção reúne, de forma qualitativa, as escolhas de projeto que instanciam o Algoritmo 1 para o problema específico deste trabalho: a busca da arquitetura de uma PINN. Os valores concretos correspondentes a cada escolha são apresentados na Seção 5.2, junto com os demais parâmetros do experimento.

Cada indivíduo representa uma arquitetura candidata de PINN, codificada pelo cromossomo definido na Equação 2.11: o número de camadas ocultas L_i e o número de neurônios n_i , igual em todas as camadas ocultas. Os pesos e vieses da rede não fazem parte do cromossomo; são obtidos, para cada arquitetura candidata, por meio de treinamento com o otimizador Adam (Kingma; Ba, 2014), conforme discutido na Seção 2.3.8. Ou seja, o único parâmetro efetivamente evoluído pelo algoritmo genético é a arquitetura da rede; os pesos são tratados como consequência do treinamento de cada arquitetura, não como material genético.

A *fitness* de cada indivíduo é calculada conforme a Equação 2.12: a arquitetura candidata é treinada por um número reduzido de épocas, e sua *fitness* é a média da função de custo total nas últimas épocas desse treinamento parcial. A seleção segue a estratégia de truncamento definida na Equação 2.13, mantendo apenas os indivíduos de melhor *fitness* para a etapa seguinte.

O operador de mutação, definido na Equação 2.14, é aplicado com probabilidade 1 a cada indivíduo selecionado, gerando um descendente por indivíduo por meio de incrementos aleatórios não negativos no número de camadas e no número de neurônios por camada. Não há, portanto, uma taxa de mutação por gene no sentido clássico (Seção 2.3.5): todo indivíduo selecionado é mutado, e a variação está na magnitude aleatória, e não na probabilidade, do incremento aplicado.

A substituição segue a estratégia elitista definida na Equação 2.15: os descendentes tomam o lugar dos indivíduos de pior *fitness* da população original, enquanto os indivíduos selecionados são preservados sem alteração, formando a nova população. O critério de parada, conforme discutido na Seção 2.3.6, não é iterativo: essa nova população é reavaliada por completo, com um número de épocas maior do que o utilizado na etapa de *fitness* inicial, e a melhor rede ao final da evolução é obtida selecionando-se, dentre os indivíduos reavaliados, a arquitetura A^* de menor *fitness*.

Por fim, A^* é treinada por completo e comparada a duas arquiteturas de referência, também treinadas por completo: uma arquitetura sorteada e a melhor arquitetura de uma busca aleatória com o mesmo orçamento de épocas, seguindo a estratégia de validação apresentada na Seção 4.3. A configuração concreta desse procedimento, incluindo o tamanho da população, os limites de camadas e neurônios, os números de épocas de cada etapa e o número de rodadas experimentais realizadas, é apresentada na Seção 5.2, junto com os resultados obtidos para os três problemas físicos estudados.

3 TRABALHOS CORRELATOS

3.1 Physics Informed Neural Networks

Desde sua proposição por Raissi, Perdikaris e Karniadakis (2019), as PINNs têm sido aplicadas a uma ampla gama de problemas em mecânica dos fluidos, transferência de calor, eletromagnetismo e mecânica quântica, entre outras áreas da física e da engenharia. O survey de Karniadakis *et al.* (2021), publicado na *Nature Reviews Physics*, consolida os principais avanços da área de aprendizado de máquina informado pela física, discutindo a incorporação de leis físicas tanto na função de custo, abordagem adotada neste trabalho, quanto em outras componentes do modelo. Já a revisão de Cuomo *et al.* (2022) cataloga extensivamente variantes de PINNs propostas na literatura, evidenciando que, apesar da simplicidade conceitual das PINNs, sua implementação eficaz envolve diversas escolhas de projeto não triviais, entre elas, a arquitetura da rede neural, foco central deste trabalho.

3.2 Neuroevolução em Redes Neurais e PINNs

A neuroevolução é uma área de pesquisa consolidada no contexto de redes neurais tradicionais, sendo utilizada tanto para otimizar pesos quanto para evoluir topologias de redes (Stanley *et al.*, 2019). Akut e Kulkarni (2019) demonstram a aplicação de algoritmos genéticos para o projeto automático de arquiteturas de *deep learning*, evidenciando ganhos de desempenho em comparação a arquiteturas definidas manualmente.

No contexto específico das PINNs, a combinação com algoritmos evolutivos ainda é um tema recente, mas já conta com resultados promissores na literatura. Kaplarević-Mališić *et al.* (2023) propõem o PINN/GA, um *framework* de algoritmo genético com operadores de seleção, cruzamento e mutação especialmente adaptados para a busca de arquitetura e hiperparâmetros de PINNs. Os autores validam a abordagem em dois problemas: o problema de Stefan unidimensional e a equação de Helmholtz. Eles relatam que o erro médio da PINN diminui cerca de uma ordem de grandeza ao longo do processo evolutivo, que o projeto computacional distribuído proposto acelera a busca de hiperparâmetros em quase duas ordens de grandeza, e que o PINN/GA apresenta desempenho superior ao de uma busca aleatória paralela e ao do método *Tree of Parzen Estimators* do Hyperopt, executados no mesmo ambiente computacional.

Sung *et al.* (2023) propõem um conjunto de cinco problemas de referência (*benchmark*): a equação de convecção–difusão, o movimento de projéteis, a equação de Korteweg–de Vries e as equações de Burgers linear e não linear, para comparar o desempenho de algoritmos de neuroevolução (CMA-ES e xNES+NAG) frente ao gradiente descendente estocástico (SGD) no treinamento de PINNs. Os autores validam a eficácia da neuroevolu-

ção por meio da comparação do MSE de teste e do grau de conformidade da solução com a equação diferencial (resíduo físico) entre os métodos, demonstrando que os algoritmos evolutivos podem superar o gradiente descendente na garantia do cumprimento das leis físicas pela solução aprendida.

Fora do contexto das PINNs, a literatura de busca de arquitetura neural (*Neural Architecture Search*) mostra que a busca aleatória é uma referência forte e frequentemente difícil de superar. Li e Talwalkar (2020) mostram que uma busca aleatória com o mesmo orçamento é competitiva com métodos de busca de arquitetura bem mais sofisticados, e defendem que toda nova estratégia de busca seja comparada com ela, o que motivou a inclusão dessa referência na avaliação deste trabalho.

Por fim, o *survey* de Wong *et al.* (2026) consolida esses e outros trabalhos relacionados, apontando a combinação entre neuroevolução e PINNs como uma das fronteiras mais promissoras da área de aprendizado de máquina científico. Constata-se, entretanto, que esses trabalhos utilizam a neuroevolução para treinar os pesos da PINN (Sung *et al.*, 2023) ou executam uma busca evolutiva completa, com muitas gerações e um grande orçamento computacional distribuído (Kaplarević-Mališić *et al.*, 2023). Este trabalho investiga uma variante mínima e de baixo custo, com uma única geração guiada por treinamentos parciais, e mede, em múltiplas rodadas independentes e reproduzíveis, se ela seleciona arquiteturas superiores tanto a uma escolha aleatória quanto a uma busca aleatória com o mesmo orçamento, o que caracteriza a contribuição desta monografia.

4 METODOLOGIA

4.1 Visão Geral do Método

Este trabalho propõe a aplicação do conceito de neuroevolução à definição da arquitetura de PINNs, compondo duas etapas distintas no processo de modelagem: (i) uma etapa de busca evolutiva, na qual um algoritmo genético explora o espaço de possíveis arquiteturas, número de camadas ocultas e número de neurônios por camada, avaliando cada candidata por meio de um treinamento parcial e abreviado como PINN; e (ii) uma etapa de treinamento completo, na qual a arquitetura selecionada ao final do processo evolutivo é treinada integralmente como PINN, sendo então comparada a duas arquiteturas de referência, também treinadas integralmente: uma arquitetura escolhida aleatoriamente e a melhor arquitetura encontrada por uma busca aleatória com o mesmo orçamento de épocas da busca evolutiva. Essa composição permite avaliar não apenas se a PINN evoluída resolve adequadamente o problema físico proposto, mas também se o processo de busca evolutiva é, de fato, capaz de identificar arquiteturas superiores a uma escolha aleatória, sem a necessidade de treinar exaustivamente todas as arquiteturas candidatas, e se esse ganho se deve aos operadores evolutivos ou apenas ao fato de avaliar mais de uma candidata.

4.2 Algoritmo Genético

A representação do indivíduo, a função de *fitness*, os operadores evolutivos e o algoritmo de neuroevolução empregados neste trabalho para a busca da arquitetura da PINN foram formalizados nas Seções 2.3.3 a 2.3.6 do Capítulo 2 (Equações 2.11 a 2.15, e Algoritmo 1). Esse procedimento é aplicado de forma idêntica aos três problemas físicos estudados neste trabalho (pêndulo, transferência de calor em uma barra unidimensional e corda oscilante): em todos os casos, a população é composta pelas arquiteturas candidatas de PINN, codificadas conforme a Equação 2.11, e a busca segue exatamente o Algoritmo 1.

Ao final da busca evolutiva, a arquitetura A^* selecionada pelo Algoritmo 1 é treinada com o total de E_{total} épocas de treinamento de uma PINN ($E_{total} > E_{term} > E_{fit}$), sendo comparada às arquiteturas de referência descritas a seguir, também treinadas com o mesmo número total de épocas. Os valores concretos adotados para o tamanho da população, os limites de camadas e neurônios, o número de épocas de cada etapa e o número de rodadas experimentais realizadas, idênticos para os três problemas, são apresentados na Seção 5.2.

4.3 Estratégia de Validação

4.3.1 Braços de Comparação

Cada rodada do experimento compara três braços, todos terminando com o treinamento completo, por E_{total} épocas, da arquitetura escolhida:

1. **GA**: a arquitetura A^* selecionada pelo Algoritmo 1, cuja busca consome $M \cdot E_{fit} + M \cdot E_{term}$ épocas de treinamento;
2. **Aleatória**: uma única arquitetura sorteada com os mesmos limites de camadas e de neurônios da população inicial do GA, sem nenhuma busca;
3. **Busca aleatória**: M arquiteturas sorteadas com os mesmos limites, cada uma treinada por $(E_{fit} + E_{term})$ épocas e ordenada pela mesma *fitness* (Equação 2.12); a melhor é a escolhida. Essa busca consome exatamente o mesmo orçamento de épocas do GA.

O braço aleatório mede o ganho de realizar qualquer busca em relação a não realizar nenhuma. A busca aleatória com o mesmo orçamento, por sua vez, isola a contribuição dos operadores evolutivos (seleção, mutação e reavaliação) do efeito de simplesmente avaliar mais de uma candidata, seguindo a recomendação da literatura de busca de arquitetura neural de comparar toda nova estratégia de busca com essa referência (Li; Talwalkar, 2020).

4.3.2 Métricas

Como os três problemas estudados possuem solução exata conhecida (Capítulo 5), a métrica principal é o erro L^2 relativo da PINN treinada em relação a essa solução,

$$\varepsilon_{L^2} = \frac{\|\hat{u} - u\|_2}{\|u\|_2} = \sqrt{\frac{\sum_j (\hat{u}(x_j) - u(x_j))^2}{\sum_j u(x_j)^2}}, \quad (4.1)$$

calculado em uma malha de avaliação independente dos pontos usados no treinamento. Essa métrica mede diretamente o que se deseja de uma PINN, que é aproximar a solução do problema físico, e não depende da escala das parcelas da função de custo. A métrica secundária é o custo final, definido como a média da função de custo total nas últimas 100 épocas do treinamento completo, que suaviza as oscilações do otimizador. Em ambas, valores menores indicam um resultado melhor.

4.3.3 Análise Estatística

O experimento completo é repetido em várias rodadas independentes. Em cada rodada, os três braços são executados com sementes derivadas da mesma semente-mestra,

de modo que a comparação entre eles é **pareada**: compara-se o GA de uma rodada com as referências da mesma rodada. Para cada comparação entre o GA e uma referência, calculam-se:

- o número de rodadas em que o GA obteve o menor erro (vitórias), com o intervalo de confiança de 95% de Wilson para a taxa de vitória (Wilson, 1927);
- um teste binomial exato unilateral, cuja hipótese nula é a de que o GA e a referência têm a mesma probabilidade de vencer (0,5);
- um teste de postos sinalizados de Wilcoxon unilateral (Wilcoxon, 1945), aplicado às diferenças pareadas $d = \ln(\varepsilon_{\text{ref}}/\varepsilon_{\text{GA}})$, que considera não apenas o sinal, mas também o tamanho de cada diferença, e não exige que os dados sigam uma distribuição normal;
- a razão mediana $\varepsilon_{\text{ref}}/\varepsilon_{\text{GA}}$, com intervalo de confiança de 95% obtido por *bootstrap* (Efron, 1979); valores acima de 1 favorecem o GA.

O logaritmo é utilizado porque os erros das PINNs variam em várias ordens de grandeza: com ele, um erro duas vezes menor e um erro duas vezes maior têm o mesmo peso, com sinais opostos. Como são realizadas seis comparações para cada teste (três problemas e duas referências), os p -valores são ajustados pelo procedimento de Holm (Holm, 1979), que mantém em 5% a probabilidade de ao menos um falso positivo no conjunto das comparações. Adota-se o nível de significância de 5% para os p -valores ajustados.

Por fim, para verificar a hipótese em que se apoia o método, a de que o custo após um treinamento parcial ordena as candidatas de forma semelhante ao erro, calcula-se a correlação de postos de Spearman (Spearman, 1904) entre a *fitness* e o erro L^2 relativo das candidatas avaliadas na triagem do GA.

4.4 Reprodutibilidade

Todas as decisões aleatórias de uma rodada r (população inicial, mutações, arquiteturas de referência e inicialização dos pesos de cada rede) são derivadas de uma semente-mestra $1000 + r$, por meio do gerador `SeedSequence` da biblioteca NumPy, e as sementes derivadas são registradas junto com os resultados da rodada. Os treinamentos utilizam algoritmos determinísticos do PyTorch, de modo que cada rodada pode ser reproduzida individualmente no mesmo *hardware* e com as mesmas versões de *software*, também registradas. A configuração de cada experimento é mantida em arquivos de texto, separados do código, e cada rodada armazena a configuração completa, o histórico da função de custo de todos os treinamentos e os pesos finais das redes. O código, os resultados e um conjunto de testes automatizados, que verificam, entre outros pontos, que as soluções

exatas satisfazem as equações, que as derivadas calculadas pela diferenciação automática coincidem com as analíticas e que as redes utilizadas possuem derivada segunda não nula, estão disponíveis publicamente¹.

¹ <https://github.com/GMCavalheri/MBA-PINN-Neuroevolution>

5 AVALIAÇÃO EXPERIMENTAL

5.1 Setup Computacional

O método foi implementado na linguagem Python 3.12, com as bibliotecas PyTorch 2.11 (com CUDA 12.8), NumPy, SciPy, pandas e Matplotlib. O PyTorch foi utilizado para construir e treinar as redes neurais e as PINNs e para calcular, por diferenciação automática (`torch.autograd.grad`), as derivadas da rede em relação às suas entradas que compõem o resíduo das equações diferenciais. O SciPy foi utilizado nos testes estatísticos, o pandas na organização dos resultados e o Matplotlib na geração de todas as figuras deste trabalho.

Os experimentos foram executados em um computador portátil com processador Intel Core i7-10750H (2,60 GHz, 6 núcleos) e placa de vídeo NVIDIA GeForce RTX 2060 Mobile (6 GB). Como as redes avaliadas são pequenas, o tempo de treinamento na GPU é dominado pelo disparo de muitas operações pequenas, e não pelo cálculo em si. Por isso, cada passo de treinamento, que inclui o cálculo da função de custo, das derivadas da rede em relação às entradas, da retropropagação e da atualização do Adam, é capturado uma única vez em um grafo CUDA (*CUDA graph*) e depois reexecutado, o que reduziu o tempo por época de 3 a 13 vezes em um teste preliminar. Também foi avaliado o treinamento em precisão mista (*automatic mixed precision*, com números de ponto flutuante de 16 bits), mas ele foi mais lento e aumentou o erro L^2 relativo em mais de 10% nos problemas de EDP, que dependem de derivadas segundas precisas; por isso, todos os treinamentos utilizam precisão simples (32 bits). Três rodadas foram executadas em paralelo na GPU.

5.2 Configuração da Neuroevolução

A configuração da neuroevolução, descrita qualitativamente na Seção 2.3.10, foi aplicada de forma idêntica aos três problemas físicos estudados neste trabalho (pêndulo, transferência de calor e corda oscilante) e está resumida na Tabela 1. Adotou-se uma população de tamanho $M = 10$ arquiteturas, geradas aleatoriamente na etapa de inicialização da população, respeitando as seguintes restrições: número de camadas ocultas entre 1 e 10, e número de neurônios por camada entre 1 e 20. Todas as redes utilizam a tangente hiperbólica como função de ativação, inicialização de Xavier (Géron, 2019) e o otimizador Adam (Kingma; Ba, 2014).

Na etapa de função de *fitness*, cada uma das $M = 10$ arquiteturas foi treinada por $E_{fit} = 1.000$ épocas, equivalente a 5% do total de $E_{total} = 20.000$ épocas de treinamento, e sua *fitness* $f(A_i)$ foi calculada como a média da função de custo total nas últimas $W = 100$ épocas (Equação 2.12). Na fase de seleção, selecionaram-se as $K = 5$ arquiteturas com

menor *fitness*.

Na etapa de mutação, o operador μ foi aplicado às $K = 5$ arquiteturas selecionadas, gerando $K = 5$ novos descendentes, com incrementos Δn_i de 0, 1 ou 2 neurônios, aplicados igualmente a todas as camadas ocultas, e ΔL_i de 0 ou 1 camada adicional. Os limites de 1 a 10 camadas e de 1 a 20 neurônios valem apenas para a população inicial; um descendente pode, portanto, ultrapassá-los em uma camada ou em até dois neurônios. Na etapa de substituição, os descendentes substituíram as $K = 5$ arquiteturas de pior desempenho da população original, mantendo o tamanho da população em $M = 10$ indivíduos.

Por fim, o critério de parada estabeleceu que as 10 arquiteturas resultantes da etapa de substituição fossem treinadas por $E_{term} = 2.000$ épocas (10% do total de épocas de treinamento), selecionando-se como solução final A^* a arquitetura de menor *fitness* ao término dessa etapa. A busca completa consome, portanto, $10 \times 1.000 + 10 \times 2.000 = 30.000$ épocas, o equivalente a cerca de 1,5 treinamento completo.

A arquitetura A^* foi então treinada com o total de $E_{total} = 20.000$ épocas de treinamento, sendo comparada, conforme a estratégia de validação apresentada na Seção 4.3, a uma arquitetura sorteada e à melhor de 10 arquiteturas sorteadas e treinadas por 3.000 épocas cada (busca aleatória), que consome o mesmo orçamento de 30.000 épocas, ambas também treinadas com E_{total} épocas. Esse experimento completo foi repetido em 35 rodadas independentes, com sementes-mestras de 1000 a 1034, para cada um dos três problemas físicos estudados.

Tabela 1 – Configuração dos experimentos, idêntica para os três problemas.

Parâmetro	Valor
Tamanho da população (M)	10
Indivíduos selecionados (K)	5
Camadas ocultas (população inicial)	1 a 10
Neurônios por camada (população inicial)	1 a 20
Incremento de mutação, camadas (ΔL)	0 ou 1
Incremento de mutação, neurônios por camada (Δn)	0, 1 ou 2
Épocas de triagem (E_{fit})	1.000
Épocas de reavaliação (E_{term})	2.000
Épocas do treinamento completo (E_{total})	20.000
Janela da <i>fitness</i> (W)	últimas 100 épocas
Busca aleatória (referência)	10 arquiteturas $\times$ 3.000 épocas
Função de ativação	tangente hiperbólica
Rodadas independentes por problema	35 (sementes 1000 a 1034)

Fonte: Elaborada pelo autor.

5.3 Pêndulo

No trabalho atual, o pêndulo simples unidimensional, sem amortecimento e sem força externa, na aproximação de pequenos ângulos, tem seu movimento descrito pela EDO abaixo

$$\frac{d^2x}{dt^2} = \ddot{x} = -\omega_0^2 \times x; \quad (5.1)$$

$$g(x, t) = \ddot{x} + \omega_0^2 x = 0, \quad (5.2)$$

em que ω_0 é a frequência angular e $\ddot{x}$ é a derivada segunda de x em relação a t . Essa é uma equação que já foi muito estudada na literatura e possui a seguinte solução analítica (Thornton; Marion, 2004):

$$x(t) = A \times \sin(\omega_0 t - \delta), \quad (5.3)$$

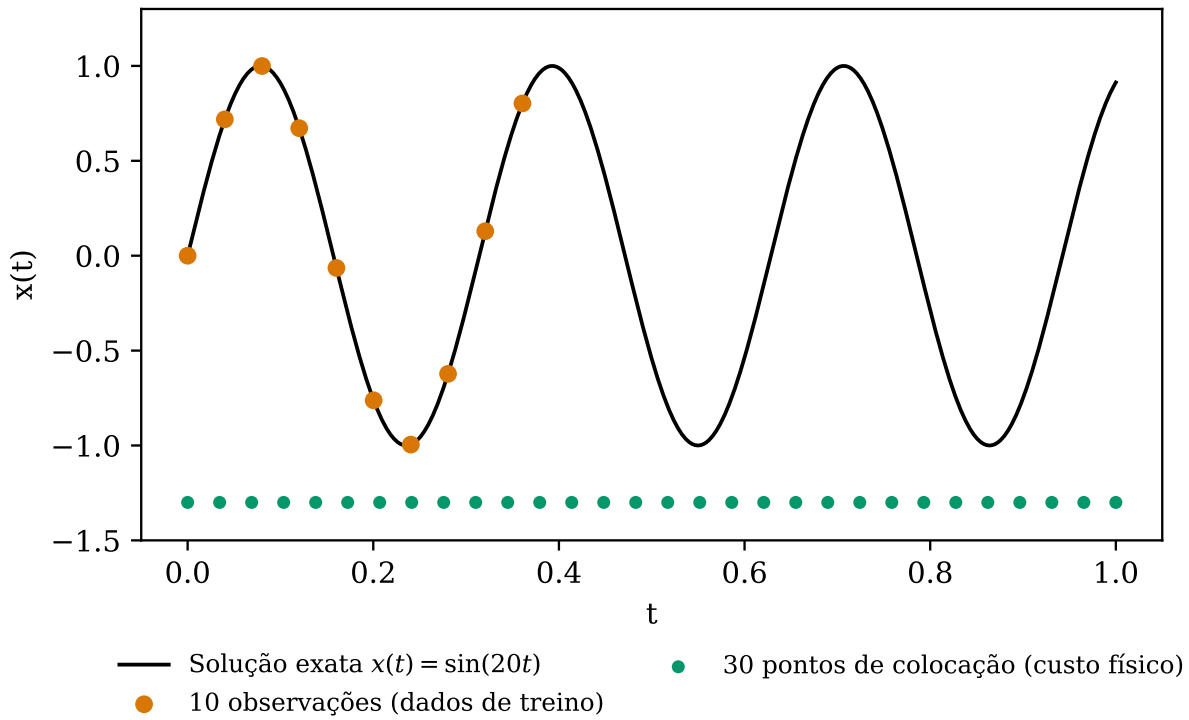
em que A é a amplitude de oscilação e δ é a constante de fase. Para o pêndulo unidimensional, o número de neurônios e atributos alvo é 1, o valor da posição do pêndulo no eixo x , $x(t)$. O número de neurônios e atributos de entrada é 1, o valor do tempo t , que é a variável independente.

Utilizaram-se os parâmetros $A = 1$, $\delta = 0$, $\omega_0 = 20$ e $t \in [0, 1]$, de modo que a solução exata é $x(t) = \sin(20t)$, com cerca de 3,2 oscilações no intervalo. O domínio do tempo foi dividido em 500 pontos igualmente espaçados, dos quais apenas 10, igualmente espaçados no início do domínio ($t < 0,4$), são utilizados como dados de treinamento da função de custo dos dados. A função de custo física é calculada em 30 pontos de colocação igualmente espaçados em todo o intervalo $[0, 1]$, como mostra a Figura 2. A função de custo da PINN é $\mathcal{L}_{\text{dados}} + \lambda \mathcal{L}_{\text{física}}$, com $\lambda = 10^{-4}$: como o resíduo da Equação 5.2 tem escala dada por $\omega_0^2 = 400$, esse peso equilibra a contribuição das duas parcelas. O erro L^2 relativo é calculado em 1.000 pontos igualmente espaçados em $[0, 1]$.

5.3.1 Rede Neural Padrão e PINN

Para ilustrar a necessidade de se usar uma arquitetura PINN, em vez de uma arquitetura padrão, foram treinadas duas redes neurais com a mesma arquitetura estrutural para resolver o problema do pêndulo: uma leva em consideração o MSE dos dados e a contribuição física do problema, caracterizando-a como uma PINN, e a outra utiliza apenas o MSE dos dados, caracterizando-a como padrão. As duas redes têm 3 camadas ocultas com 32 neurônios totalmente conectados em cada uma delas, um neurônio na camada de entrada e um na camada de saída, ativação tangente hiperbólica e a mesma inicialização dos pesos (semente fixa). A entrada da rede recebe o tempo t , variável independente, e a

Figura 2 – Solução exata do pêndulo, dados de treinamento e pontos de colocação da função de custo física.



Fonte: Elaborada pelo autor.

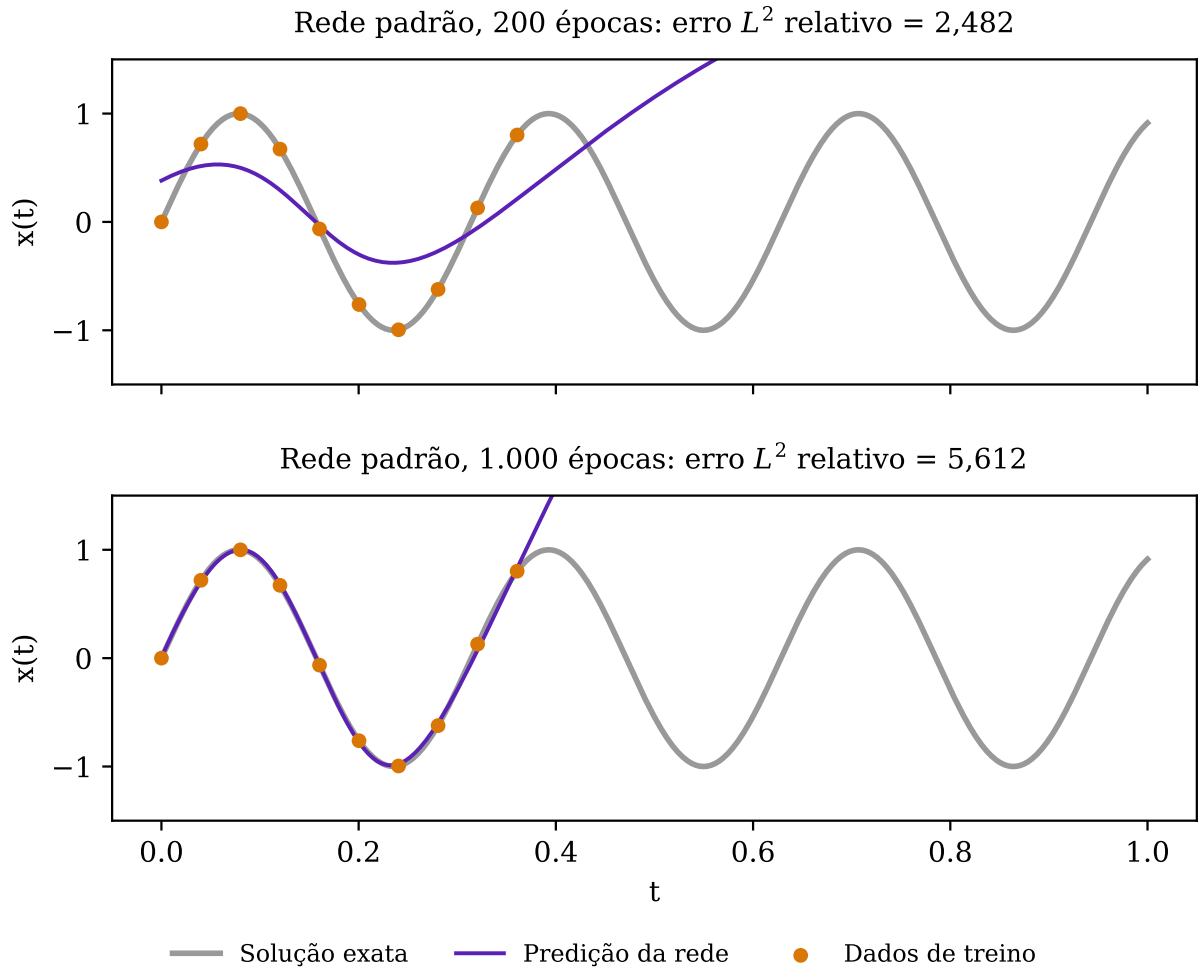
saída fornece a posição x , variável dependente. O algoritmo de otimização utilizado é o Adam (Kingma; Ba, 2014).

Realizou-se o treinamento da rede neural padrão, com apenas a função de custo MSE, durante 1.000 épocas, com taxa de aprendizado de 10^{-3} , obtendo os resultados mostrados na Figura 3. Após 200 épocas, a rede ainda não se ajustou aos dados; após 1.000 épocas, ela interpola os 10 pontos de treinamento, mas, fora deles, segue uma tendência que não tem relação com o fenômeno físico, com erro L^2 relativo de 5,6 em todo o intervalo.

Realizou-se, em seguida, o treinamento da PINN, com a mesma arquitetura e os mesmos dados de treinamento, porém adicionando a função de custo física, com taxa de aprendizado de 10^{-4} e 20.000 épocas. A biblioteca PyTorch possui a função `autograd.grad`, que calcula, por diferenciação automática, as derivadas da saída da rede em relação às suas entradas; aplicada duas vezes, ela fornece $\ddot{x}$, utilizada no resíduo da Equação 5.2. A Figura 4 mostra as previsões da PINN nas épocas 6.000, 12.000, 18.000 e 20.000.

Nota-se pelas imagens que a PINN necessita de um número maior de épocas para interpolar os dados de treinamento, em comparação à rede neural padrão. Entretanto, o ganho em generalização, devido à função de custo física, é muito mais importante e ocorre em paralelo com a interpolação dos dados: ao final do treinamento, a PINN reproduz a

Figura 3 – Valores preditos pela rede neural padrão após 200 e 1.000 épocas de treinamento.



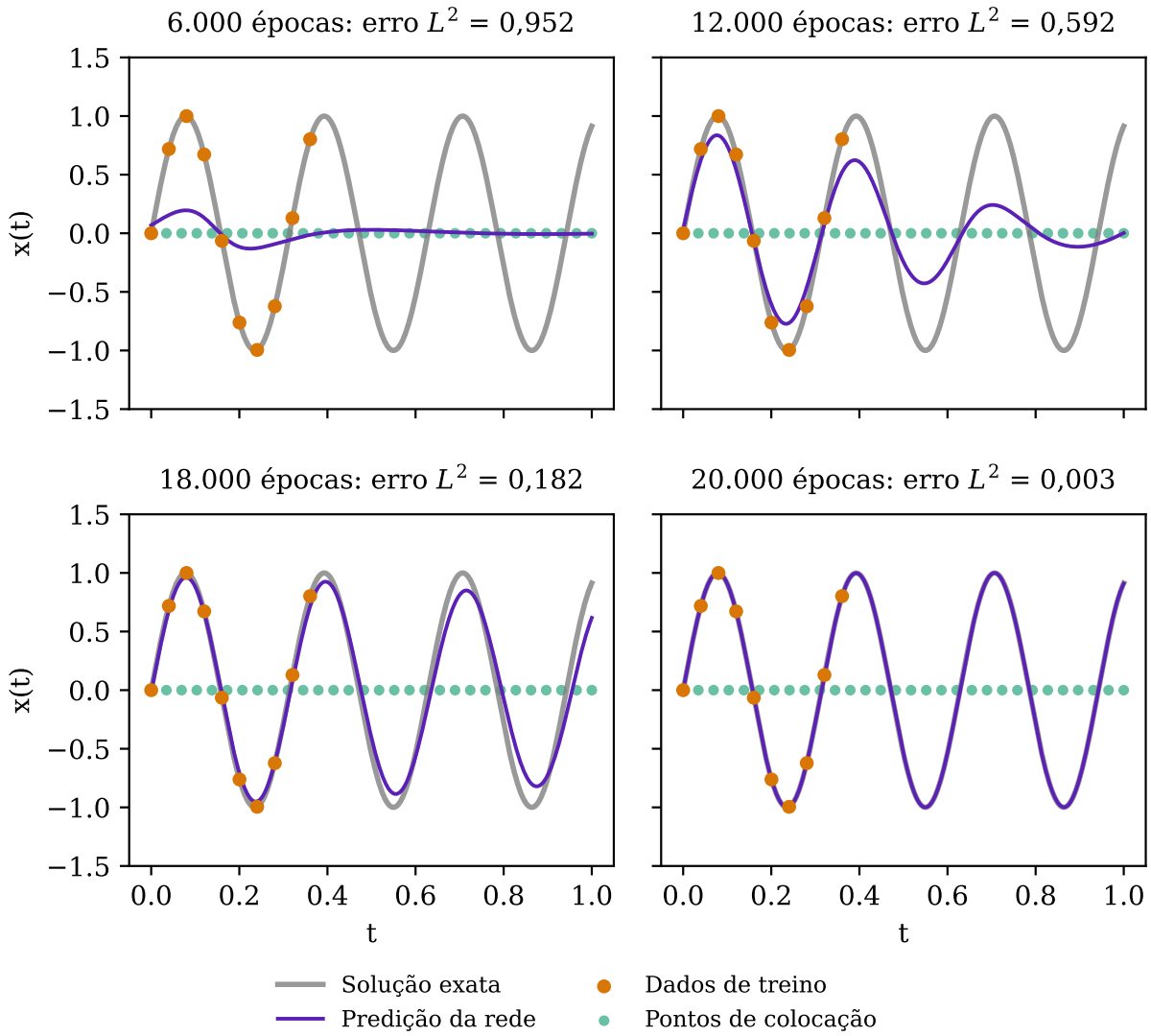
Fonte: Elaborada pelo autor.

oscilação em todo o intervalo, inclusive na região sem dados, com erro L^2 relativo de 0,003, cerca de 2.000 vezes menor do que o da rede padrão.

A não generalização do modelo padrão decorre de sua função de custo ser referente apenas aos dados do problema, e não ao tipo de fenômeno físico que está sendo analisado. Uma função de custo de MSE, utilizada isoladamente, não consegue identificar qual problema físico está sendo tratado: se os dados fossem provenientes de qualquer outra área da física, ela tentaria apenas minimizá-los, sem capturar a dinâmica subjacente ao sistema.

A Figura 5 mostra a evolução das funções de custo das duas redes. Na rede padrão, o custo praticamente não diminui nas primeiras 200 épocas e, logo em seguida, tem uma redução muito acentuada, seguida de uma redução lenta até a época 1.000, o que mostra que o modelo se adapta rapidamente aos dados de treinamento. Na PINN, as duas parcelas da função de custo competem entre si. A parcela física cai rapidamente nas primeiras épocas, pois uma rede com saída próxima de zero satisfaz trivialmente a Equação 5.2; à

Figura 4 – Valores preditos pela PINN após 6.000, 12.000, 18.000 e 20.000 épocas de treinamento.



Fonte: Elaborada pelo autor.

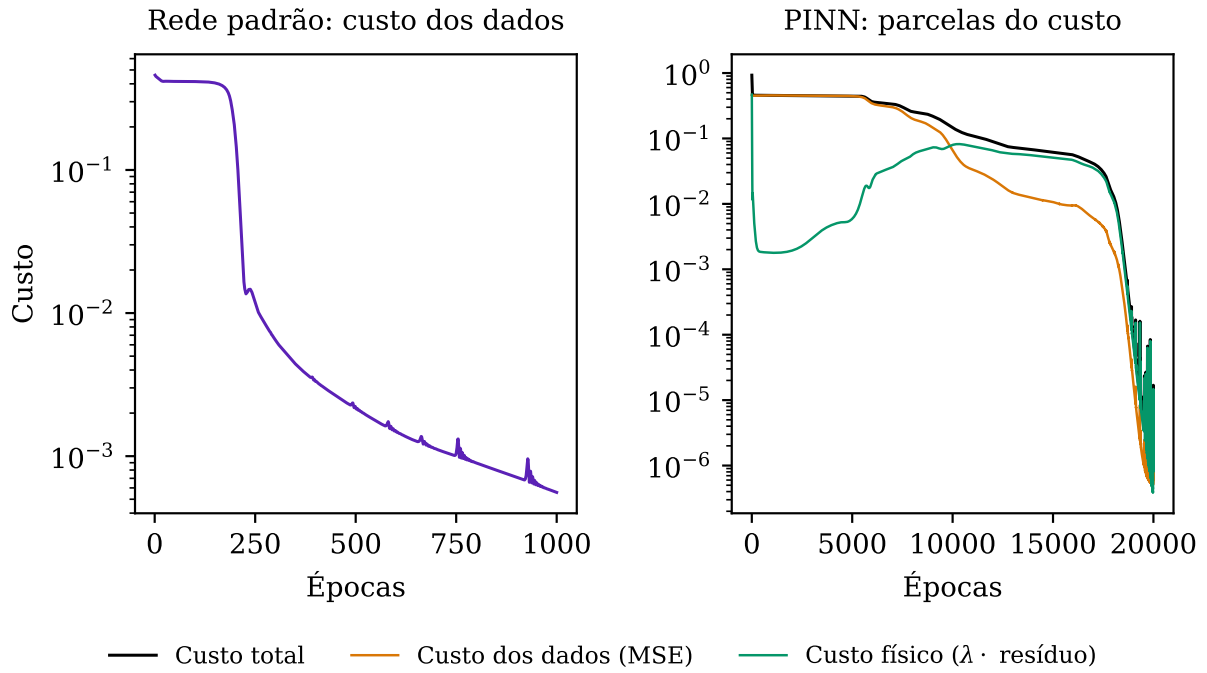
medida que a rede passa a se ajustar aos dados e a oscilar, a parcela física volta a crescer, até que, a partir de cerca de 17.000 épocas, a rede encontra uma oscilação que satisfaz simultaneamente os dados e a equação, e as duas parcelas caem várias ordens de grandeza.

5.3.2 Exemplo de uma Rodada de Neuroevolução

Para ilustrar o funcionamento do Algoritmo 1, a Tabela 2 apresenta a rodada 5 do pêndulo, cuja PINN obtida com a arquitetura do GA teve o erro mediano entre as 35 rodadas. Das 10 arquiteturas da população inicial, treinadas por 1.000 épocas, as 5 de menor *fitness* foram selecionadas, e cada uma gerou um descendente por mutação.

Observa-se que, após 1.000 épocas, as *fitness* das candidatas diferem apenas na

Figura 5 – Funções de custo da rede neural padrão e das parcelas da função de custo da PINN, em função das épocas de treinamento.



Fonte: Elaborada pelo autor.

Tabela 2 – As 5 arquiteturas selecionadas na triagem da rodada 5 do pêndulo, com sua *fitness* após E_{fit} épocas, o descendente gerado por mutação e a *fitness* de ambos após E_{term} épocas. As arquiteturas são indicadas como (camadas ocultas, neurônios por camada).

Selecionada	$Fitness(E_{fit})$	$Fitness(E_{term})$	Descendente	$Fitness(E_{term})$
(7, 15)	0,4514	0,2420	(8, 16)	0,3636
(10, 3)	0,4521	0,4512	(11, 5)	0,4565
(2, 15)	0,4559	0,4536	(2, 16)	0,4566
(10, 2)	0,4562	0,4213	(11, 4)	0,3304
(6, 12)	0,4566	0,4552	(7, 14)	0,4438

Fonte: Elaborada pelo autor.

terceira casa decimal: no pêndulo, com taxa de aprendizado de 10^{-4} , as redes ainda estão no platô inicial do treinamento (Figura 5), e a triagem distingue as candidatas com pouca margem. Após a reavaliação com 2.000 épocas, as diferenças se tornam maiores, e a melhor arquitetura foi a própria arquitetura (7, 15), selecionada na triagem, e não um de seus descendentes. A Tabela 3 compara essa arquitetura com as duas referências da mesma rodada.

Nessa rodada, a arquitetura do GA teve erro cerca de 3,7 vezes menor do que

Tabela 3 – Arquiteturas dos três braços na rodada 5 do pêndulo e erro L^2 relativo da PINN após o treinamento completo.

Braço	Camadas ocultas	Neurônios por camada	Erro L^2 relativo
GA	7	15	$5,8 \times 10^{-3}$
Aleatória	7	7	$2,1 \times 10^{-2}$
Busca aleatória	7	17	$3,5 \times 10^{-4}$

Fonte: Elaborada pelo autor.

a arquitetura sorteada, mas maior do que a da busca aleatória, que encontrou uma arquitetura de mesma profundidade e um pouco mais larga. Os resultados das 35 rodadas são apresentados na Seção 5.6.

5.4 Transferência de calor em uma barra unidimensional

A transferência de calor em uma barra unidimensional é descrita pela EDP abaixo (Hahn; Özisik, 2012)

$$\alpha \frac{\partial^2 u}{\partial x^2} = \frac{\partial u}{\partial t}; \quad (5.4)$$

$$g(x, u, t) = \alpha \frac{\partial^2 u}{\partial x^2} - \frac{\partial u}{\partial t} = 0, \quad (5.5)$$

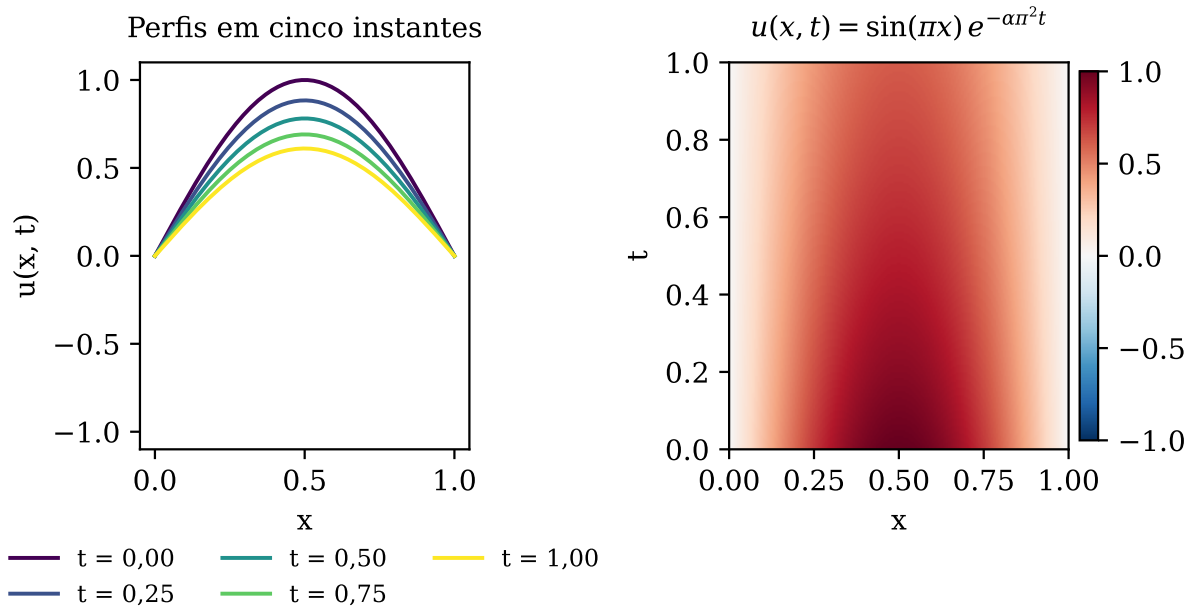
em que u é a temperatura na posição x e no tempo t , e α é a difusividade térmica, que mede a velocidade com que o calor se propaga pelo material. Essa é uma das equações mais utilizadas e estudadas em física do estado sólido e termodinâmica. Utilizou-se o problema na forma adimensional, com $x, t \in [0, 1]$ e $\alpha = 0,05$, condição inicial $u(x, 0) = \sin(\pi x)$, que representa um ponto quente no centro da barra, e temperatura nula nas extremidades, $u(0, t) = u(1, t) = 0$. Para essas condições, o método de separação de variáveis fornece a solução exata

$$u(x, t) = \sin(\pi x) e^{-\alpha \pi^2 t}, \quad (5.6)$$

na qual o perfil inicial mantém sua forma e decai exponencialmente com o tempo, como mostra a Figura 6.

A entrada da rede recebe a posição x e o tempo t , variáveis independentes, e a saída fornece a temperatura u , variável dependente. A função de ativação de todas as camadas é a tangente hiperbólica, o otimizador é o Adam (Kingma; Ba, 2014) e a taxa de aprendizado é 10^{-3} . O resíduo da Equação 5.5 é calculado em uma malha uniforme de pontos de colocação com 100 pontos em x e 250 pontos em t , totalizando 25.000 pares

Figura 6 – Solução exata do problema de transferência de calor utilizado neste trabalho: perfis de temperatura em cinco instantes e mapa de $u(x, t)$.



Fonte: Elaborada pelo autor.

(x, t) . A função de custo é a soma de quatro parcelas de erro quadrático médio, com pesos iguais: o resíduo nos pontos de colocação, a condição inicial (100 pontos em $t = 0$) e as duas condições de contorno (250 pontos em cada extremidade). Não há dados observados além das condições inicial e de contorno. O erro L^2 relativo é calculado em uma malha independente de 201×201 pontos.

5.5 Corda oscilante

O movimento de uma corda oscilante, com deslocamento transversal u ao longo de uma dimensão espacial x , é descrito pela equação da onda (Thornton; Marion, 2004)

$$c^2 \frac{\partial^2 u}{\partial x^2} = \frac{\partial^2 u}{\partial t^2}; \quad (5.7)$$

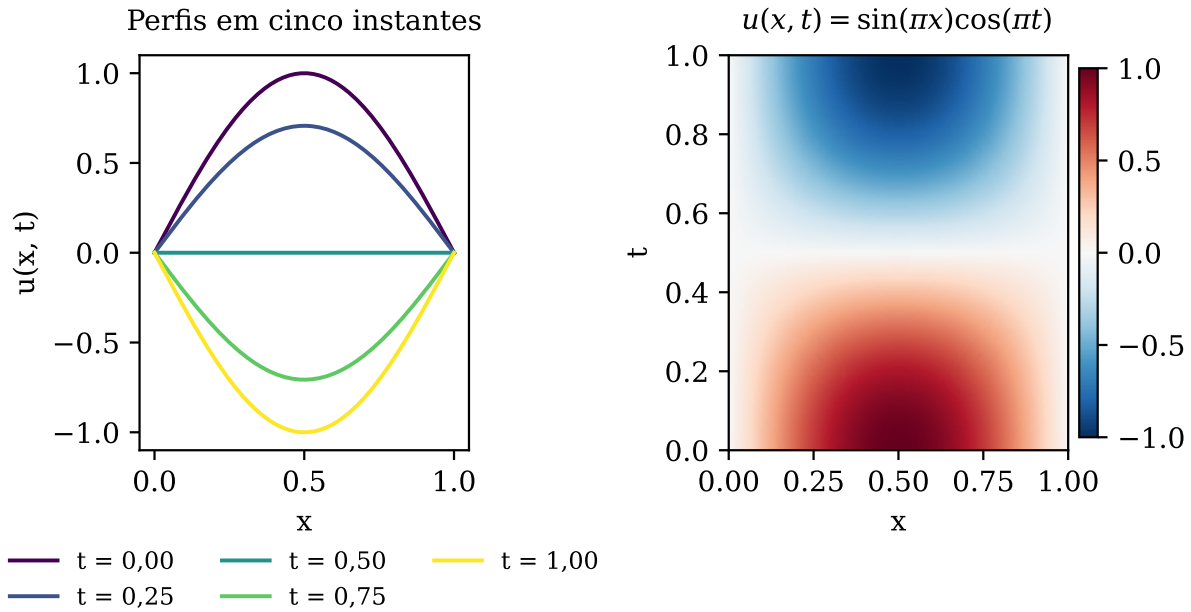
$$g(x, u, t) = c^2 \frac{\partial^2 u}{\partial x^2} - \frac{\partial^2 u}{\partial t^2} = 0, \quad (5.8)$$

em que u é o deslocamento da corda na posição x e no tempo t , e c é a velocidade de propagação da onda no meio. Utilizou-se o problema na forma adimensional, com $x, t \in [0, 1]$ e $c = 1$, deslocamento inicial $u(x, 0) = \sin(\pi x)$, velocidade inicial nula, $\partial u / \partial t(x, 0) = 0$, e extremidades fixas, $u(0, t) = u(1, t) = 0$. Para essas condições, a solução exata é a onda estacionária

$$u(x, t) = \sin(\pi x) \cos(\pi t), \quad (5.9)$$

que realiza meio período de oscilação no intervalo, partindo do deslocamento máximo em $t = 0$, passando pelo repouso em $t = 0,5$ e chegando ao deslocamento oposto em $t = 1$, como mostra a Figura 7.

Figura 7 – Solução exata do problema da corda oscilante utilizado neste trabalho: perfis de deslocamento em cinco instantes e mapa de $u(x, t)$.



Fonte: Elaborada pelo autor.

A rede, a malha de pontos de colocação, o otimizador e a taxa de aprendizado são os mesmos do problema de transferência de calor. A função de custo acrescenta uma quinta parcela, referente à condição de velocidade inicial nula, calculada por diferenciação automática nos 100 pontos de $t = 0$.

5.6 Resultados da Neuroevolução

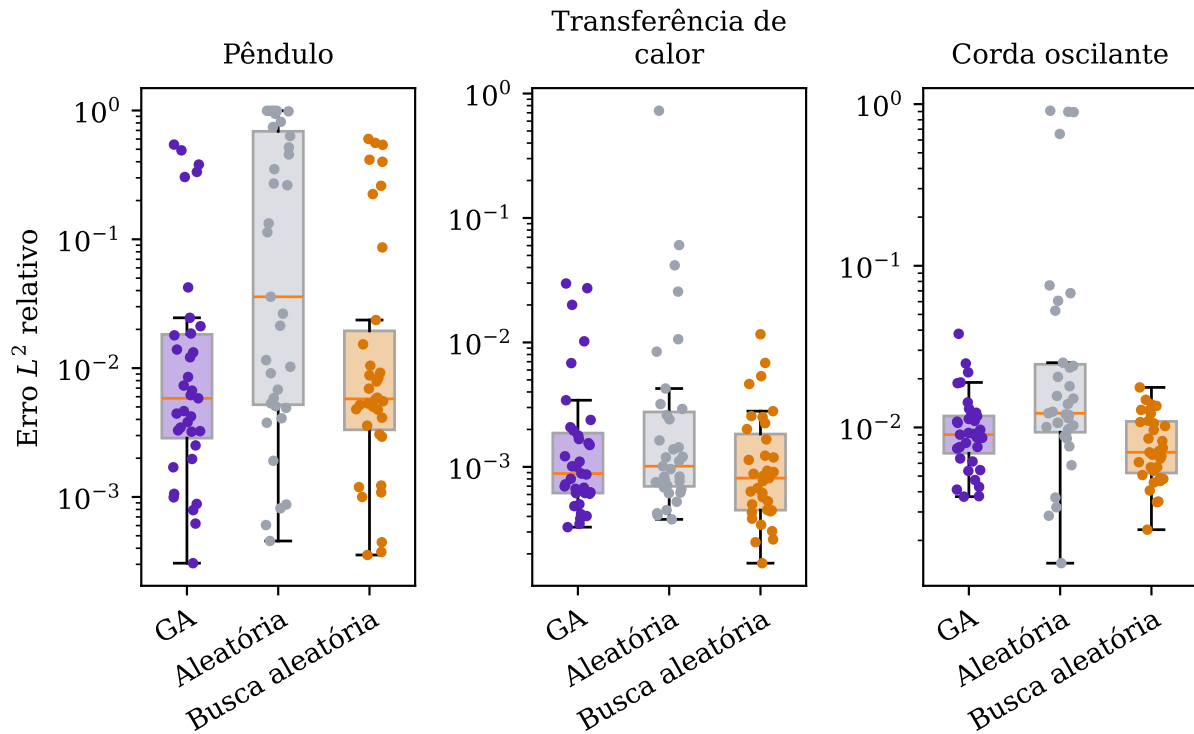
As 105 rodadas (35 por problema) foram concluídas, e nenhum treinamento divergiu. A Tabela 4 apresenta a mediana do erro L^2 relativo da rede final de cada braço, e a Figura 8 mostra a distribuição desse erro nas 35 rodadas de cada problema. Os três braços obtiveram soluções precisas na maioria das rodadas: a Figura 9 mostra a PINN obtida com a arquitetura do GA na rodada de erro mediano de cada problema, cujo erro absoluto máximo é de 0,004 na transferência de calor e de 0,018 na corda oscilante, para uma solução de amplitude 1.

Tabela 4 – Mediana do erro L^2 relativo e mediana do número de parâmetros treináveis da rede final de cada braço (35 rodadas por problema), e correlação de Spearman ρ_s entre a *fitness* e o erro L^2 relativo das 350 candidatas triadas pelo GA em cada problema.

Problema	Mediana do erro L^2 relativo			Mediana de parâmetros			ρ_s
	GA	Aleatória	Busca aleatória	GA	Aleatória	Busca aleatória	
Pêndulo	$5,8 \times 10^{-3}$	0,036	$5,8 \times 10^{-3}$	1.966	505	1.726	0,79
Transferência de calor	$8,8 \times 10^{-4}$	$1,0 \times 10^{-3}$	$8,1 \times 10^{-4}$	1.947	517	1.217	0,93
Corda oscilante	$9,0 \times 10^{-3}$	0,012	$7,0 \times 10^{-3}$	881	517	681	0,99

Fonte: Elaborada pelo autor.

Figura 8 – Erro L^2 relativo da rede final de cada braço nas 35 rodadas de cada problema, em escala logarítmica. As caixas mostram os quartis, e cada ponto é uma rodada.

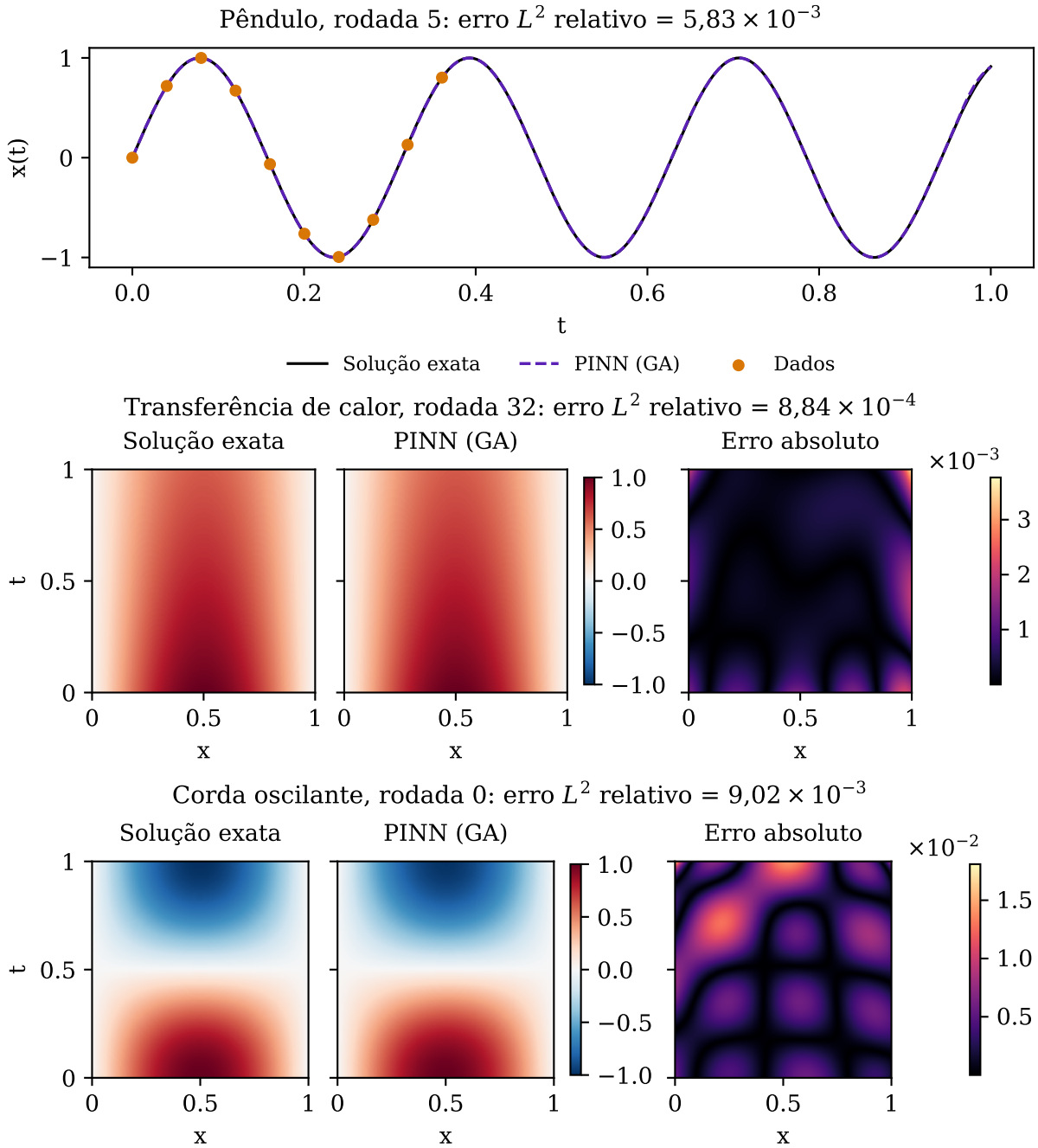


Fonte: Elaborada pelo autor.

A principal diferença entre os braços está nas falhas, isto é, nas rodadas em que a PINN não reproduziu a solução (erro relativo acima de 0,5): a arquitetura aleatória falhou em 11 rodadas do pêndulo, em uma da transferência de calor e em quatro da corda oscilante, enquanto o GA falhou em uma única rodada do pêndulo e a busca aleatória em três, também no pêndulo. O pêndulo é o problema mais exigente, porque a PINN precisa extrapolar a oscilação muito além do intervalo observado, e redes com poucos parâmetros não conseguem fazê-lo dentro de 20.000 épocas: das 11 arquiteturas aleatórias

que falharam no pêndulo, 10 tinham no máximo 106 parâmetros treináveis, como (1, 2), (3, 1) e (4, 5), contra uma mediana de 613 parâmetros entre as que não falharam.

Figura 9 – PINN obtida com a arquitetura do GA na rodada de erro mediano de cada problema. Acima: pêndulo, com as dez observações. Meio e abaixo: solução exata, predição da PINN e erro absoluto na transferência de calor e na corda oscilante.



Fonte: Elaborada pelo autor.

5.6.1 GA versus Arquitetura Aleatória

A Tabela 5 apresenta as comparações pareadas entre o GA e cada referência. Contra a arquitetura aleatória, o GA obteve erro menor em 23, 21 e 26 das 35 rodadas do pêndulo, da transferência de calor e da corda oscilante, respectivamente, e a mediana da razão entre o erro da arquitetura aleatória e o do GA foi de 4,6, 1,5 e 1,8. Após a correção de Holm, a melhora é estatisticamente significativa na corda oscilante pelos dois testes ($p_B = 0,018$ e $p_W = 0,003$) e no pêndulo pelo teste de Wilcoxon ($p_W = 0,027$).

Tabela 5 – Comparações pareadas do GA com cada referência nas 35 rodadas de cada problema, no erro L^2 relativo. Vitórias: rodadas em que o GA obteve erro menor, com o intervalo de Wilson de 95% da taxa de vitória; p_B : teste binomial unilateral; p_W : teste de postos sinalizados de Wilcoxon unilateral; ambos ajustados por Holm sobre as seis comparações. Razão mediana: mediana do erro da referência dividido pelo erro do GA, com intervalo *bootstrap* de 95% (valores acima de 1 favorecem o GA). Vitórias (custo): vitórias na métrica secundária.

Problema	Referência	Vitórias	Wilson 95%	p_B	p_W	Razão mediana [IC 95%]	Vitórias (custo)
Pêndulo	Aleatória	23/35	49%–79%	0,224	0,027	4,62 [0,74; 29,76]	22/35
	Busca aleatória	20/35	41%–72%	0,749	0,881	1,58 [0,31; 4,02]	17/35
Transferência de calor	Aleatória	21/35	44%–74%	0,621	0,736	1,49 [0,57; 2,25]	18/35
	Busca aleatória	16/35	30%–62%	1,000	1,000	0,93 [0,65; 1,30]	10/35
Corda oscilante	Aleatória	26/35	58%–86%	0,018	$3,3 \times 10^{-3}$	1,78 [1,24; 2,59]	22/35
	Busca aleatória	12/35	21%–51%	1,000	1,000	0,76 [0,64; 1,10]	11/35

Fonte: Elaborada pelo autor.

No pêndulo, o teste binomial isolado não é significativo após a correção ($p_B = 0,22$, ou 0,045 antes da correção): o GA perde em 12 rodadas, mas essas derrotas são pequenas, enquanto várias de suas vitórias são de ordens de grandeza, contra arquiteturas aleatórias que falharam. As 11 maiores diferenças entre os erros, em módulo, foram todas vitórias do GA. O teste de Wilcoxon, que considera o tamanho das diferenças pareadas, capta essa assimetria; o binomial, que considera apenas o sinal, não. Na transferência de calor, nenhuma comparação é significativa: quase todas as arquiteturas com ativação tangente hiperbólica resolveram esse problema suave e difusivo com erro da ordem de 10^{-3} , de modo que a escolha da arquitetura fez pouca diferença.

5.6.2 GA versus Busca Aleatória com o Mesmo Orçamento

Contra a busca aleatória com o mesmo orçamento, o GA não foi superior em nenhum dos problemas. Ele venceu em 20, 16 e 12 das 35 rodadas, com razões medianas de 1,58, 0,93 e 0,76, e todos os p -valores ajustados são altos, de 0,75 a 1. Na corda oscilante, a comparação tende a favorecer a busca aleatória, que obteve erro menor em 23 das 35 rodadas; um teste de Wilcoxon no sentido oposto, que não fazia parte da análise planejada e deve ser visto como exploratório, resulta em $p = 0,022$ antes e $p = 0,066$ após a correção

de Holm sobre os três problemas, ou seja, não significativo. A métrica secundária leva às mesmas conclusões: o GA obteve custo final menor do que a arquitetura aleatória em 22, 18 e 22 rodadas, e do que a busca aleatória em apenas 17, 10 e 11 rodadas.

A Figura 10 mostra a função de custo ao longo do treinamento completo dos três braços na rodada de erro mediano do GA em cada problema. O custo oscila fortemente com o Adam, com picos curtos de duas ordens de grandeza ou mais, o que justifica tomar a *fitness* como uma média das últimas épocas. Na rodada do pêndulo, a arquitetura aleatória, com 7 camadas ocultas de 7 neurônios, converge muito mais devagar do que as duas obtidas por busca, enquanto na rodada da transferência de calor as três redes terminam com custos e erros semelhantes.

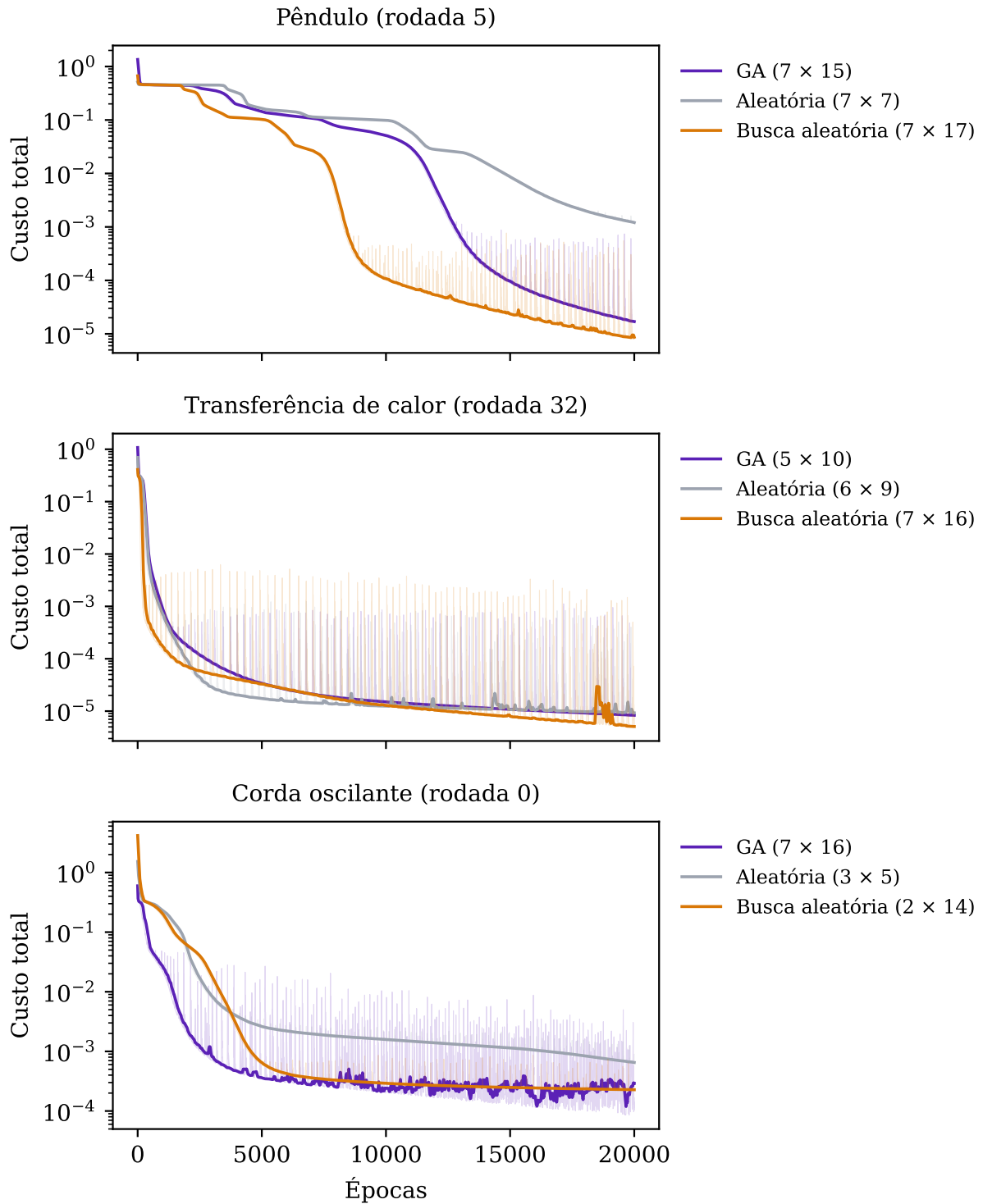
5.6.3 O Treinamento Parcial como Indicador

O método se apoia na hipótese de que o custo após um treinamento curto ordena as candidatas como o erro as ordenaria. Nas 350 candidatas triadas pelo GA em cada problema (10 por rodada), a correlação de Spearman entre a *fitness* após E_{fit} épocas e o erro L^2 relativo no mesmo ponto do treinamento foi de 0,79 no pêndulo, 0,93 na transferência de calor e 0,99 na corda oscilante (Tabela 4), todas com $p < 10^{-75}$. O valor mais baixo no pêndulo é coerente com o observado na Tabela 2: após 1.000 épocas, as candidatas ainda estão no platô inicial do treinamento, e suas *fitness* são muito próximas. Entre as 105 redes finais de cada problema, a correlação entre o custo final e o erro foi de 0,91, 0,51 e 0,67, respectivamente. Portanto, o custo da PINN é um critério confiável para ordenar as candidatas neste cenário, sem recorrer à solução exata, embora se torne menos informativo entre redes que já são todas precisas.

5.6.4 Arquiteturas Seleccionadas e Custo Computacional

As arquiteturas escolhidas pelo GA foram as maiores dos três braços, com mediana de 1.966, 1.947 e 881 parâmetros treináveis, contra 1.726, 1.217 e 681 na busca aleatória (Tabela 4), como esperado de uma mutação que apenas acrescenta camadas e neurônios. Na corda oscilante, as duas buscas preferiram redes rasas e largas, com medianas de 4 e 3 camadas ocultas de 16 neurônios, enquanto nos outros problemas escolheram redes mais profundas, de 7 a 9 camadas ocultas. Como o GA treina redes maiores na reavaliação, sua busca levou de 25% a 35% mais tempo do que a busca aleatória com o mesmo número de épocas, com medianas de 177, 733 e 1.078 segundos por rodada, contra 142, 546 e 808 segundos. As 105 rodadas levaram cerca de 23 horas no total, com três rodadas em paralelo na GPU: 2,4 horas para o pêndulo, 9,2 horas para a transferência de calor e 11,6 horas para a corda oscilante.

Figura 10 – Função de custo total ao longo do treinamento completo dos três braços na rodada de erro mediano do GA em cada problema. As linhas finas mostram o custo em cada época, e as grossas, sua mediana móvel em 200 épocas. As legendas indicam as arquiteturas como camadas ocultas $\times$ neurônios por camada.



Fonte: Elaborada pelo autor.

5.7 Discussão

Os resultados confirmam parte da hipótese inicial deste trabalho. Uma busca guiada por treinamentos parciais, com custo de cerca de 1,5 treinamento completo, escolhe arquiteturas de PINN mais precisas e, sobretudo, mais confiáveis do que uma escolha aleatória: o GA evitou quase todas as falhas da arquitetura aleatória, e, na rodada típica, o erro da arquitetura aleatória foi de 1,5 a 4,6 vezes o do GA (razão mediana), com significância estatística no pêndulo e na corda oscilante.

Os operadores evolutivos, porém, não acrescentaram valor em relação à busca aleatória com o mesmo orçamento. Uma explicação plausível é a forma como o orçamento é distribuído. A busca aleatória ordena 10 candidatas independentes com 3.000 épocas cada, enquanto o GA as ordena com apenas 1.000 épocas e gasta o restante do orçamento reavaliando as cinco melhores e cinco descendentes que diferem delas em no máximo uma camada e dois neurônios por camada. Com um indicador tão confiável quanto o medido na Seção 5.6.3, uma avaliação mais longa de candidatas diversas parece valer mais do que pequenas variações locais das melhores. Esse resultado concorda com a literatura de busca de arquitetura neural, na qual a busca aleatória é uma referência forte e frequentemente difícil de superar (Li; Talwalkar, 2020), e contrasta com o do PINN/GA (Kaplarević-Mališić *et al.*, 2023), que supera a busca aleatória utilizando muitas gerações, cruzamento e um orçamento computacional muito maior.

Algumas limitações devem ser consideradas. Primeiro, executa-se uma única geração, e a mutação apenas aumenta a rede, o que enviesava a busca para arquiteturas maiores e limita sua exploração; um GA com várias gerações e mutação bidirecional poderia se comportar de forma diferente. Segundo, o espaço de busca tem apenas dois genes, com largura uniforme, e a taxa de aprendizado e os demais hiperparâmetros foram fixados. Terceiro, os três problemas são lineares, de baixa dimensão e com soluções suaves; na transferência de calor, em particular, a arquitetura quase não importa, e problemas mais difíceis são necessários para discriminar entre estratégias de busca. Por fim, as rodadas na GPU são reprodutíveis no mesmo *hardware* e com as mesmas versões de *software*, que são registradas em cada rodada, mas podem diferir ligeiramente em outras plataformas.

6 CONCLUSÕES

Este trabalho teve como objetivo introduzir o conceito de neuroevolução na definição da arquitetura de *Physics Informed Neural Networks* (PINNs), buscando reduzir o custo computacional associado à escolha manual ou exaustiva da arquitetura de rede neural, discutido na introdução deste trabalho a partir do exemplo da Llama 3 (Grattafiori *et al.*, 2024). Para isso, foram estudados três problemas físicos clássicos com solução exata, descritos por equações diferenciais ordinárias e parciais: o pêndulo simples, a transferência de calor em uma barra unidimensional e a corda oscilante.

Inicialmente, comparou-se o desempenho de uma rede neural padrão, treinada apenas com a função de custo MSE, com o de uma PINN, treinada também com uma função de custo física baseada na equação diferencial do pêndulo. Os resultados demonstraram que, embora a PINN necessite de um número maior de épocas para interpolar os dados de treinamento em comparação à rede neural padrão, apenas a abordagem PINN foi capaz de generalizar e extrapolar corretamente o comportamento do sistema físico para além do intervalo de dados observados, com erro L^2 relativo de 0,003, contra 5,6 da rede padrão, validando a necessidade da incorporação da física ao processo de treinamento em cenários com poucos dados iniciais.

Em seguida, aplicou-se a neuroevolução, por meio de um Algoritmo Genético de uma única geração que evolui o número de camadas ocultas e de neurônios por camada, para selecionar a arquitetura da PINN em cada um dos três problemas, comparando o resultado a uma arquitetura escolhida aleatoriamente e a uma busca aleatória com o mesmo orçamento de épocas. Em 35 rodadas reprodutíveis por problema, a arquitetura selecionada pelo GA obteve erro L^2 relativo menor do que a arquitetura aleatória em 23, 21 e 26 rodadas, com razão mediana de 1,5 a 4,6 entre o erro da arquitetura aleatória e o do GA, e evitou quase todas as falhas da escolha aleatória. A melhora foi estatisticamente significativa no pêndulo e na corda oscilante, mas não na transferência de calor, um problema suave que quase qualquer arquitetura resolveu com precisão. Esses resultados indicam que o processo de busca, mesmo utilizando apenas uma fração do total de épocas de treinamento para avaliar cada arquitetura candidata, identifica arquiteturas superiores a uma escolha aleatória.

O GA, porém, não superou a busca aleatória com o mesmo orçamento em nenhum dos problemas. Isso indica que, com uma única geração, o ganho vem de avaliar várias candidatas com treinamentos parciais, e não dos operadores evolutivos em si. Contribui para essa conclusão a verificação de que o custo do treinamento parcial é um indicador confiável do erro das candidatas, com correlação de Spearman de 0,79 a 0,99: com um indicador assim, avaliar candidatas diversas por mais tempo, como faz a busca aleatória,

mostrou-se tão eficaz quanto refinar localmente as melhores, como faz o GA.

Cabe destacar algumas limitações deste trabalho. Os três problemas físicos abordados são relativamente simples, de baixa dimensão e descritos por equações diferenciais lineares e bem estabelecidas na literatura; não se avaliou, portanto, o comportamento do método proposto em problemas de maior complexidade, dimensionalidade ou não linearidade. Além disso, o espaço de busca da neuroevolução foi restrito ao número de camadas ocultas e ao número de neurônios por camada de uma arquitetura MLP com largura uniforme, a mutação apenas aumenta a rede e o algoritmo executa uma única geração. Por fim, esta versão revisada corrige os experimentos de transferência de calor e de corda oscilante da versão original, cujas redes utilizavam a ativação ReLU, que anula as derivadas segundas presentes nessas equações; todos os experimentos foram refeitos com a tangente hiperbólica, soluções exatas e sementes fixas, e o código e os resultados estão disponíveis publicamente.

Como trabalhos futuros, sugere-se, em primeiro lugar, comparar o GA com a busca aleatória e com outras metaheurísticas e algoritmos de otimização, como *Particle Swarm Optimization* (PSO), Estratégias Evolutivas (*Evolution Strategies*, como o CMA-ES) e Otimização Bayesiana, sob o mesmo orçamento computacional, em problemas mais difíceis e não lineares, como as equações de Burgers e de Navier–Stokes, nos quais a escolha da arquitetura tende a importar mais. Sugere-se também estudar a distribuição do orçamento entre o número de candidatas e a duração de sua avaliação, e executar várias gerações com mutação bidirecional e cruzamento. Sugere-se, ainda, ampliar o espaço de busca para além do número de camadas e neurônios, incorporando larguras diferentes entre as camadas, funções de ativação, taxa de aprendizado e técnicas de regularização, e explorar outros tipos de arquitetura além da MLP, como Redes Neurais Convolucionais (CNN), Redes Neurais Recorrentes do tipo *Long Short-Term Memory* (LSTM) e *Gated Recurrent Unit* (GRU), e arquiteturas baseadas em *Transformer*.

REFERÊNCIAS

- AKUT, R.; KULKARNI, S. Neuroevolution: Using genetic algorithm for optimal design of deep learning models. *In: IEEE. 2019 IEEE International Conference on Electrical, Computer and Communication Technologies (ICECCT)*. [S.l.: s.n.], 2019. p. 1–6.
- BRIGATO, L.; IOCCHI, L. A close look at deep learning with small data. *In: IEEE. 2020 25th international conference on pattern recognition (ICPR)*. [S.l.: s.n.], 2021. p. 2490–2497.
- CUOMO, S. *et al.* Scientific machine learning through physics-informed neural networks: Where we are and what’s next. **Journal of Scientific Computing**, v. 92, n. 3, p. 88, 2022.
- EFRON, B. Bootstrap methods: another look at the jackknife. **The Annals of Statistics**, v. 7, n. 1, p. 1–26, 1979.
- EIBEN, A. E.; SMITH, J. E. **Introduction to Evolutionary Computing**. 2. ed. [S.l.: s.n.]: Springer, 2015.
- GÉRON, A. **Hands-On Machine Learning with Scikit-Learn, Keras, and TensorFlow**. 2. ed. Sebastopol: O’Reilly Media, 2019.
- GOLDBERG, D. E. **Genetic Algorithms in Search, Optimization, and Machine Learning**. [S.l.: s.n.]: Addison-Wesley, 1989.
- GRATTAFIORI, A. *et al.* The Llama 3 herd of models. **arXiv preprint arXiv:2407.21783**, 2024.
- HAHN, D. W.; ÖZISIK, M. N. **Heat conduction**. [S.l.: s.n.]: John Wiley & Sons, 2012.
- HOLLAND, J. H. Genetic algorithms. **Scientific American**, v. 267, n. 1, p. 66–73, 1992.
- HOLM, S. A simple sequentially rejective multiple test procedure. **Scandinavian Journal of Statistics**, v. 6, n. 2, p. 65–70, 1979.
- KAPLAREVIĆ-MALIŠIĆ, A. *et al.* Identifying optimal architectures of physics-informed neural networks by evolutionary strategy. **Applied Soft Computing**, v. 146, p. 110646, 2023.
- KARNIADAKIS, G. E. *et al.* Physics-informed machine learning. **Nature Reviews Physics**, v. 3, n. 6, p. 422–440, 2021.
- KINGMA, D. P.; BA, J. Adam: A method for stochastic optimization. **arXiv preprint arXiv:1412.6980**, 2014.
- LI, L.; TALWALKAR, A. Random search and reproducibility for neural architecture search. *In: Proceedings of the 35th Uncertainty in Artificial Intelligence Conference*. [S.l.: s.n.]: PMLR, 2020. (Proceedings of Machine Learning Research, v. 115), p. 367–377.

RAISSI, M.; PERDIKARIS, P.; KARNIADAKIS, G. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. **Journal of Computational Physics**, v. 378, p. 686–707, 2019. ISSN 0021-9991. Disponível em: <https://www.sciencedirect.com/science/article/pii/S0021999118307125>.

SPEARMAN, C. The proof and measurement of association between two things. **The American Journal of Psychology**, v. 15, n. 1, p. 72–101, 1904.

STANLEY, K. O. *et al.* Designing neural networks through neuroevolution. **Nature Machine Intelligence**, Nature Publishing Group UK London, v. 1, n. 1, p. 24–35, 2019.

STANLEY, K. O.; MIIKKULAINEN, R. Evolving neural networks through augmenting topologies. **Evolutionary Computation**, MIT Press, v. 10, n. 2, p. 99–127, 2002.

SUNG, N. W. Y. *et al.* Neuroevolution of physics-informed neural nets: Benchmark problems and comparative results. *In: Proceedings of the Companion Conference on Genetic and Evolutionary Computation (GECCO)*. [S.l.: s.n.], 2023.

THORNTON, S. T.; MARION, J. **Classical dynamics of particles and systems**. 5. ed. Belmont: Thomson Brooks/Cole, 2004. 672 p.

WIDROW, B.; HOFF, M. E. Adaptive switching circuits. *In: IRE WESCON Convention Record*. [S.l.: s.n.], 1960. v. 4, p. 96–104.

WILCOXON, F. Individual comparisons by ranking methods. **Biometrics Bulletin**, v. 1, n. 6, p. 80–83, 1945.

WILSON, E. B. Probable inference, the law of succession, and statistical inference. **Journal of the American Statistical Association**, v. 22, n. 158, p. 209–212, 1927.

WONG, J. C. *et al.* Evolutionary optimization of physics-informed neural networks: Evo-PINN frontiers and opportunities. **IEEE Computational Intelligence Magazine**, 2026. ArXiv:2501.06572.

YAO, X. Evolving artificial neural networks. **Proceedings of the IEEE**, IEEE, v. 87, n. 9, p. 1423–1447, 1999.